```
# ===== File: app/main.py =====
"""
FastAPI 主入口
航空发动机装配排产仿真系统 - AeroEngine Assembly Scheduling Simulation
北京航空航天大学 (Beihang University)
"""
import os
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import HTMLResponse, JSONResponse
from app.api import config, process, simulation, results
# 创建FastAPI应用实例
app = FastAPI(
    title="航空发动机装配排产仿真系统",
    description="AeroEngine Assembly Scheduling Simulation - Beihang University",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
)
# CORS中间件配置 - 允许跨域访问
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# 静态文件服务 - 前端资源
frontend_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), "frontend")
if os.path.exists(frontend_dir):
    app.mount("/static", StaticFiles(directory=frontend_dir), name="static")
    # 挂载css和js目录到根路径（支持模块化前端）
    css_dir = os.path.join(frontend_dir, "css")
    js_dir = os.path.join(frontend_dir, "js")
    if os.path.exists(css_dir):
        app.mount("/css", StaticFiles(directory=css_dir), name="css")
    if os.path.exists(js_dir):
        app.mount("/js", StaticFiles(directory=js_dir), name="js")
# 注册API路由
app.include_router(config.router, prefix="/api/config", tags=["配置管理"])
app.include_router(process.router, prefix="/api/process", tags=["工艺流程"])
app.include_router(simulation.router, prefix="/api/simulation", tags=["仿真控制"])
app.include_router(results.router, prefix="/api/results", tags=["结果查询"])
@app.get("/", response_class=HTMLResponse)
async def root():
    """
    根路径 - 返回前端页面
    """
    index_path = os.path.join(frontend_dir, "index.html")
```

```
    if os.path.exists(index_path):
        with open(index_path, encoding="utf-8") as f:
            return HTMLResponse(content=f.read())
    return HTMLResponse(content="""
    <!DOCTYPE html>
    <html>
    <head>
        <meta charset="UTF-8">
        <title>航空发动机装配排产仿真系统</title>
        <style>
            body {
                font-family: 'Microsoft YaHei', sans-serif;
                background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
                color: #eee;
                display: flex;
                justify-content: center;
                align-items: center;
                height: 100vh;
                margin: 0;
            }
            .container {
                text-align: center;
                padding: 40px;
                background: rgba(255,255,255,0.05);
                border-radius: 16px;
                border: 1px solid rgba(255,255,255,0.1);
            }
            h1 { color: #00d4ff; margin-bottom: 10px; }
            h2 { color: #888; font-weight: normal; margin-bottom: 30px; }
            a {
                display: inline-block;
                padding: 12px 24px;
                background: #00d4ff;
                color: #1a1a2e;
                text-decoration: none;
                border-radius: 8px;
                margin: 10px;
                font-weight: bold;
            }
            a:hover { background: #00b8e6; }
            .watermark {
                margin-top: 40px;
                color: rgba(255,255,255,0.3);
                font-size: 12px;
            }
        </style>
    </head>
    <body>
        <div class="container">
            <h1>  航空发动机装配排产仿真系统</h1>
```

```
            <h2>AeroEngine Assembly Scheduling Simulation</h2>
            <p>后端服务已启动，前端页面待部署</p>
            <a href="/docs">  API文档 (Swagger)</a>
            <a href="/redoc">  API文档 (ReDoc)</a>
            <div class="watermark">
                北京航空航天大学 | Beihang University<br>
                AeroEngine Scheduling System v1.0.0
            </div>
        </div>
    </body>
    </html>
    """)
@app.get("/health")
async def health_check():
    """
    健康检查接口
    """
    return JSONResponse(content={
        "status": "healthy",
        "version": "1.0.0",
        "service": "AeroEngine Assembly Scheduling Simulation"
    })
@app.on_event("startup")
async def startup_event():
    """
    应用启动事件
    """
    # 创建结果存储目录
    results_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), "results")
    os.makedirs(results_dir, exist_ok=True)
    print("  航空发动机装配排产仿真系统启动成功!")
    print("  API文档: http://localhost:8000/docs")
    print("  前端界面: http://localhost:8000")
@app.on_event("shutdown")
async def shutdown_event():
    """
    应用关闭事件
    """
    print("  航空发动机装配排产仿真系统已关闭")
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000, reload=True)
# ===== File: app/__init__.py =====
"""
航空发动机装配排产仿真系统
AeroEngine Assembly Scheduling Simulation System
所属单位: 北京航空航天大学 (Beihang University)
版本: v1.0.0
"""
__version__ = "1.0.0"
```

```
__author__ = "Beihang University"
__description__ = "航空发动机装配排产仿真系统"
# ===== File: app/api/__init__.py =====
"""
API模块包
包含所有REST API端点的定义
模块说明:
- config.py: 配置管理接口
- process.py: 工艺流程接口
- simulation.py: 仿真控制接口
- results.py: 结果查询接口
"""
from app.api import config, process, simulation, results
__all__ = ["config", "process", "simulation", "results"]
# ===== File: app/api/config.py =====
"""
配置管理接口
提供全局配置的获取、更新和验证功能
API端点:
- GET /api/config/default: 获取默认配置
- POST /api/config/validate: 验证配置有效性
- POST /api/config/update: 更新配置
- GET /api/config/equipment-types: 获取关键设备类型列表
"""
import os
from typing import Dict, List, Any, Optional
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
import yaml
router = APIRouter()
# ============ 请求/响应模型 ============
class GlobalConfig(BaseModel):
    """全局配置模型"""
    work_hours_per_day: int = Field(default=8, ge=1, le=24, description="每日工作小时数")
    work_days_per_month: int = Field(default=22, ge=1, le=31, description="每月工作天数")
    num_workers: int = Field(default=6, ge=1, description="工人数量")
    # 关键设备配置（前端自定义）
    critical_equipment: Dict[str, int] = Field(
        default={"动平衡机": 2, "试车台": 1, "装配台": 3, "检测台": 2},
        description="关键设备及数量"
    )
    # 休息规则参数
    rest_time_threshold: float = Field(default=50.0, ge=0, description="连续工作触发休息阈值（分钟）")
    rest_duration_time: float = Field(default=5.0, ge=0, description="时间触发休息时长（分钟）")
    rest_load_threshold: int = Field(default=7, ge=1, le=10, description="负荷触发休息阈值（REBA 1-10）")
    rest_duration_load: float = Field(default=3.0, ge=0, description="负荷触发休息时长（分钟）")
    # 生产目标
    target_output: int = Field(default=3, ge=1, description="目标月产量（台）")
    @property
    def sim_time_minutes(self) -> float:
```

```
        """计算仿真总时长（分钟）"""
        return self.work_hours_per_day * 60 * self.work_days_per_month
    class Config:
        json_schema_extra = {
            "example": {
                "work_hours_per_day": 8,
                "work_days_per_month": 22,
                "num_workers": 6,
                "critical_equipment": {
                    "动平衡机": 2,
                    "试车台": 1,
                    "装配台": 3,
                    "检测台": 2
                },
                "rest_time_threshold": 50.0,
                "rest_duration_time": 5.0,
                "rest_load_threshold": 7,
                "rest_duration_load": 3.0,
                "target_output": 3
            }
        }
class APIResponse(BaseModel):
    """统一API响应格式"""
    success: bool = Field(description="请求是否成功")
    message: str = Field(description="响应消息")
    data: Optional[Any] = Field(default=None, description="响应数据")
class ConfigValidationResult(BaseModel):
    """配置验证结果"""
    valid: bool
    errors: List[str] = []
    warnings: List[str] = []
class EquipmentUpdate(BaseModel):
    """设备更新请求"""
    name: str = Field(description="设备名称")
    quantity: int = Field(ge=1, description="设备数量")
# ============ API端点 ============
@router.get("/default", response_model=APIResponse)
async def get_default_config():
    """
    获取默认配置
    返回系统默认的全局配置参数，包括：
    - 排班配置（工时/天数/工人数）
    - 关键设备配置
    - 休息规则参数
    - 生产目标
    """
    # 尝试从配置文件加载
    config_path = os.path.join(
        os.path.dirname(os.path.dirname(os.path.dirname(__file__))),
        "config", "default_config.yaml"
```

```
    )
    if os.path.exists(config_path):
        try:
            with open(config_path, "r", encoding="utf-8") as f:
                config_data = yaml.safe_load(f)
                config = GlobalConfig(**config_data)
        except Exception as e:
            config = GlobalConfig()
    else:
        config = GlobalConfig()
    return APIResponse(
        success=True,
        message="获取默认配置成功",
        data=config.model_dump()
    )
@router.post("/validate", response_model=APIResponse)
async def validate_config(config: GlobalConfig):
    """
    验证配置有效性
    检查配置参数的合理性，包括：
    - 参数范围检查
    - 设备数量检查
    - 逻辑一致性检查
    """
    errors = []
    warnings = []
    # 检查工作时间设置
    if config.work_hours_per_day > 12:
        warnings.append(f"每日工作时间 {config.work_hours_per_day} 小时可能过长")
    # 检查工人数量
    if config.num_workers < 2:
        warnings.append("工人数量过少，可能导致流水线效率低下")
    # 检查关键设备
    if not config.critical_equipment:
        errors.append("至少需要配置一种关键设备")
    for equip_name, quantity in config.critical_equipment.items():
        if quantity < 1:
            errors.append(f"设备 '{equip_name}' 数量必须大于0")
    # 检查休息规则
    if config.rest_time_threshold < config.rest_duration_time:
        warnings.append("休息触发阈值小于休息时长，可能导致频繁休息")
    # 检查仿真时间
    sim_time = config.sim_time_minutes
    if sim_time < 480:  # 少于8小时
        warnings.append(f"仿真总时长 {sim_time} 分钟较短，可能无法完成目标产量")
    # 估算产能
    estimated_capacity = sim_time / (480 * 1.5)  # 假设每台需要1.5天
    if config.target_output > estimated_capacity * 1.5:
        warnings.append(f"目标产量 {config.target_output} 台可能超出生产能力")
    result = ConfigValidationResult(
```

```
        valid=len(errors) == 0,
        errors=errors,
        warnings=warnings
    )
    return APIResponse(
        success=result.valid,
        message="配置验证通过" if result.valid else "配置存在错误",
        data=result.model_dump()
    )
@router.post("/update", response_model=APIResponse)
async def update_config(config: GlobalConfig):
    """
    更新配置
    保存配置到配置文件（可选）
    """
    # 先验证配置
    errors = []
    if config.num_workers < 1:
        errors.append("工人数量必须大于0")
    if not config.critical_equipment:
        errors.append("必须配置关键设备")
    if errors:
        return APIResponse(
            success=False,
            message="配置验证失败",
            data={"errors": errors}
        )
    # 可选：保存到文件
    # config_path = os.path.join(...)
    # with open(config_path, "w", encoding="utf-8") as f:
    #     yaml.dump(config.model_dump(), f, allow_unicode=True)
    return APIResponse(
        success=True,
        message="配置更新成功",
        data=config.model_dump()
    )
@router.get("/equipment-types", response_model=APIResponse)
async def get_equipment_types():
    """
    获取关键设备类型列表
    返回系统支持的关键设备类型，供前端下拉选择
    """
    equipment_types = [
        {"name": "动平衡机", "description": "用于转子动平衡检测", "default_quantity": 2},
        {"name": "试车台", "description": "发动机整机试车", "default_quantity": 1},
        {"name": "装配台", "description": "部件装配工作台", "default_quantity": 3},
        {"name": "检测台", "description": "质量检测工作台", "default_quantity": 2},
        {"name": "吊装设备", "description": "重型部件吊装", "default_quantity": 2},
        {"name": "清洗设备", "description": "零件清洗", "default_quantity": 1},
        {"name": "涂装设备", "description": "防护涂装", "default_quantity": 1},
```

```
    ]
    return APIResponse(
        success=True,
        message="获取设备类型成功",
        data=equipment_types
    )
@router.post("/equipment/add", response_model=APIResponse)
async def add_equipment(equipment: EquipmentUpdate, config: GlobalConfig):
    """
    添加关键设备
    """
    if equipment.name in config.critical_equipment:
        return APIResponse(
            success=False,
            message=f"设备 '{equipment.name}' 已存在"
        )
    config.critical_equipment[equipment.name] = equipment.quantity
    return APIResponse(
        success=True,
        message=f"成功添加设备 '{equipment.name}'",
        data=config.critical_equipment
    )
@router.delete("/equipment/{name}", response_model=APIResponse)
async def remove_equipment(name: str):
    """
    删除关键设备
    """
    # 这里需要从当前配置中删除
    # 实际应用中应该从持久化存储中操作
    return APIResponse(
        success=True,
        message=f"成功删除设备 '{name}'"
    )
@router.get("/sim-time-info", response_model=APIResponse)
async def get_sim_time_info(
    work_hours_per_day: int = 8,
    work_days_per_month: int = 22
):
    """
    获取仿真时间信息
    根据配置计算仿真时长相关信息
    """
    total_minutes = work_hours_per_day * 60 * work_days_per_month
    total_hours = total_minutes / 60
    return APIResponse(
        success=True,
        message="计算成功",
        data={
            "total_minutes": total_minutes,
            "total_hours": total_hours,
```

```
            "total_days": work_days_per_month,
            "minutes_per_day": work_hours_per_day * 60,
            "description": f"仿真周期：{work_days_per_month}天，每天{work_hours_per_day}小时，共{total_hours}小时"
        }
    )
# ===== File: app/api/process.py =====
"""
工艺流程接口
提供工艺流程的创建、解析、验证和管理功能
API端点:
- GET /api/process/template: 下载CSV模板
- POST /api/process/parse-csv: 解析上传的CSV
- POST /api/process/validate: 验证工艺流程DAG
- POST /api/process/save: 保存工艺流程
- GET /api/process/example: 获取示例工艺流程
"""
import os
import io
import csv
from typing import List, Dict, Any, Optional, Set
from enum import Enum
from fastapi import APIRouter, HTTPException, UploadFile, File
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
router = APIRouter()
# ============ 枚举定义 ============
class OpType(str, Enum):
    """操作类型枚举"""
    H = "H"  # 取/放 (Handling)
    A = "A"  # 装配 (Assembly)
    M = "M"  # 测量 (Measurement) - 可触发返工
    T = "T"  # 工具操作 (Tooling)
    D = "D"  # 数据记录 (Data Recording)
# 操作类型元数据
OP_TYPE_META = {
    OpType.H: {"zh": "取/放", "en": "Handling", "color": "#3B82F6", "icon": "  "},
    OpType.A: {"zh": "装配", "en": "Assembly", "color": "#10B981", "icon": "  "},
    OpType.M: {"zh": "测量", "en": "Measurement", "color": "#F59E0B", "icon": "  "},
    OpType.T: {"zh": "工具操作", "en": "Tooling", "color": "#8B5CF6", "icon": "    "},
    OpType.D: {"zh": "数据记录", "en": "Data Recording", "color": "#6B7280", "icon": "  "},
}
# ============ 数据模型 ============
class ProcessNode(BaseModel):
    """工艺节点模型"""
    step_id: str = Field(description="唯一步骤ID")
    task_name: str = Field(description="任务名称")
    op_type: OpType = Field(description="操作类型（H/A/M/T/D）")
    predecessors: str = Field(default="", description="前置依赖（分号分隔）")
    std_duration: float = Field(ge=0, description="标准工时（分钟）")
    time_variance: float = Field(default=0.0, ge=0, description="时间波动方差")
```

```
    work_load_score: int = Field(default=5, ge=1, le=10, description="REBA负荷评分")
    rework_prob: float = Field(default=0.0, ge=0, le=1, description="返工概率（仅M类有效）")
    required_workers: int = Field(default=1, ge=1, description="所需工人数")
    required_tools: List[str] = Field(default=[], description="所需工具/设备列表")
    station: str = Field(default="ST01", description="工位ID")
    # 前端坐标（用于流程图编辑器）
    x: float = Field(default=0, description="节点X坐标")
    y: float = Field(default=0, description="节点Y坐标")
    def get_predecessor_list(self) -> List[str]:
        """解析前置依赖为列表"""
        if not self.predecessors:
            return []
        return [p.strip() for p in self.predecessors.split(";") if p.strip()]
    def get_critical_equipment(self, critical_set: Set[str]) -> List[str]:
        """获取关键设备（需要排队的）"""
        return [t for t in self.required_tools if t in critical_set]
    def get_common_tools(self, critical_set: Set[str]) -> List[str]:
        """获取普通工具（无限供应）"""
        return [t for t in self.required_tools if t not in critical_set]
class ProcessDefinition(BaseModel):
    """工艺流程定义"""
    name: str = Field(default="未命名流程", description="流程名称")
    description: str = Field(default="", description="流程描述")
    nodes: List[ProcessNode] = Field(default=[], description="工艺节点列表")
    def get_node_map(self) -> Dict[str, ProcessNode]:
        """获取节点映射字典"""
        return {node.step_id: node for node in self.nodes}
class APIResponse(BaseModel):
    """统一API响应格式"""
    success: bool
    message: str
    data: Optional[Any] = None
class ValidationResult(BaseModel):
    """验证结果"""
    valid: bool
    errors: List[str] = []
    warnings: List[str] = []
    node_count: int = 0
    edge_count: int = 0
    parallel_groups: int = 0
# ============ CSV模板 ============
CSV_TEMPLATE_HEADERS = [
    "step_id",
    "task_name",
    "op_type",
    "predecessors",
    "std_duration",
    "time_variance",
    "work_load_score",
    "rework_prob",
```

```
    "required_workers",
    "required_tools",
    "station"
]
CSV_TEMPLATE_EXAMPLE = [
    ["S001", "取压气机转子", "H", "", "5", "1", "4", "0", "2", "吊装设备", "ST01"],
    ["S002", "安装前检查", "M", "S001", "10", "2", "3", "0.05", "1", "检测台", "ST02"],
    ["S003", "装配前轴承", "A", "S002", "15", "3", "6", "0", "2", "装配台", "ST03"],
    ["S004", "装配后轴承", "A", "S002", "15", "3", "6", "0", "2", "装配台", "ST03"],
    ["S005", "安装密封件", "A", "S003;S004", "8", "1.5", "5", "0", "1", "", "ST04"],
    ["S006", "动平衡测试", "M", "S005", "30", "5", "4", "0.1", "1", "动平衡机", "ST05"],
    ["S007", "记录测试数据", "D", "S006", "5", "0.5", "2", "0", "1", "", "ST06"],
    ["S008", "最终装配", "A", "S007", "20", "4", "7", "0", "2", "装配台", "ST07"],
    ["S009", "试车准备", "I", "S008", "10", "2", "5", "0", "2", "试车台", "ST08"],
    ["S010", "整机试车", "M", "S009", "60", "10", "6", "0.15", "2", "试车台", "ST08"],
]
# ============ API端点 ============
@router.get("/template")
async def download_template():
    """
    下载CSV模板
    返回工艺流程CSV模板文件，包含表头和示例数据
    """
    output = io.StringIO()
    writer = csv.writer(output)
    # 写入表头
    writer.writerow(CSV_TEMPLATE_HEADERS)
    # 写入示例数据
    for row in CSV_TEMPLATE_EXAMPLE:
        writer.writerow(row)
    output.seek(0)
    return StreamingResponse(
        io.BytesIO(output.getvalue().encode('utf-8-sig')),  # 使用BOM便于Excel识别中文
        media_type="text/csv",
        headers={
            "Content-Disposition": "attachment; filename=process_template.csv"
        }
    )
@router.post("/parse-csv", response_model=APIResponse)
async def parse_csv(file: UploadFile = File(...)):
    """
    解析上传的CSV文件
    将CSV文件解析为工艺流程定义对象
    """
    if not file.filename.endswith('.csv'):
        return APIResponse(
            success=False,
            message="请上传CSV格式文件"
        )
    try:
```

```
    # 读取文件内容
    content = await file.read()
    # 尝试不同编码
    for encoding in ['utf-8-sig', 'utf-8', 'gbk', 'gb2312']:
        try:
            text = content.decode(encoding)
            break
        except UnicodeDecodeError:
            continue
    else:
        return APIResponse(
            success=False,
            message="无法识别文件编码，请使用UTF-8编码"
        )
    # 解析CSV
    reader = csv.DictReader(io.StringIO(text))
    nodes = []
    errors = []
    for row_num, row in enumerate(reader, start=2):
        try:
            # 解析required_tools
            tools_str = row.get('required_tools', '').strip()
            tools = [t.strip() for t in tools_str.split(';') if t.strip()] if tools_str else []
            node = ProcessNode(
                step_id=row.get('step_id', '').strip(),
                task_name=row.get('task_name', '').strip(),
                op_type=OpType(row.get('op_type', 'A').strip().upper()),
                predecessors=row.get('predecessors', '').strip(),
                std_duration=float(row.get('std_duration', 0)),
                time_variance=float(row.get('time_variance', 0)),
                work_load_score=int(row.get('work_load_score', 5)),
                rework_prob=float(row.get('rework_prob', 0)),
                required_workers=int(row.get('required_workers', 1)),
                required_tools=tools,
                station=row.get('station', 'ST01').strip() or 'ST01'
            )
            nodes.append(node)
        except Exception as e:
            errors.append(f"第{row_num}行解析错误: {str(e)}")
    if errors:
        return APIResponse(
            success=False,
            message=f"CSV解析存在 {len(errors)} 个错误",
            data={"errors": errors, "parsed_count": len(nodes)}
        )
    process = ProcessDefinition(
        name=file.filename.replace('.csv', ''),
        nodes=nodes
    )
    return APIResponse(
```

```
            success=True,
            message=f"成功解析 {len(nodes)} 个工艺节点",
            data=process.model_dump()
        )
    except Exception as e:
        return APIResponse(
            success=False,
            message=f"文件解析失败: {str(e)}"
        )
@router.post("/validate", response_model=APIResponse)
async def validate_process(process: ProcessDefinition):
    """
    验证工艺流程DAG
    检查工艺流程的有效性，包括：
    - 节点ID唯一性
    - 依赖关系有效性
    - 无循环依赖
    - 有起始和结束节点
    """
    errors = []
    warnings = []
    # 构建节点映射
    node_map = process.get_node_map()
    node_ids = set(node_map.keys())
    # 检查节点ID唯一性
    if len(node_ids) != len(process.nodes):
        errors.append("存在重复的节点ID")
    # 检查依赖关系
    edge_count = 0
    for node in process.nodes:
        predecessors = node.get_predecessor_list()
        for pred_id in predecessors:
            if pred_id not in node_ids:
                errors.append(f"节点 '{node.step_id}' 的前置依赖 '{pred_id}' 不存在")
            else:
                edge_count += 1
    # 检查是否有起始节点（无前置依赖的节点）
    start_nodes = [n for n in process.nodes if not n.get_predecessor_list()]
    if not start_nodes:
        errors.append("没有找到起始节点（所有节点都有前置依赖）")
    # 检查循环依赖（简单的DFS检测）
    def has_cycle() -> bool:
        visited = set()
        rec_stack = set()
        def dfs(node_id: str) -> bool:
            visited.add(node_id)
            rec_stack.add(node_id)
            # 获取后继节点
            for n in process.nodes:
                if node_id in n.get_predecessor_list():
```

```
                    if n.step_id not in visited:
                        if dfs(n.step_id):
                            return True
                    elif n.step_id in rec_stack:
                        return True
            rec_stack.remove(node_id)
            return False
        for node in process.nodes:
            if node.step_id not in visited:
                if dfs(node.step_id):
                    return True
        return False
    if has_cycle():
        errors.append("流程图存在循环依赖")
    # 检查M类型节点的返工概率
    for node in process.nodes:
        if node.op_type == OpType.M and node.rework_prob > 0:
            if node.rework_prob > 0.5:
                warnings.append(f"节点 '{node.step_id}' 返工概率 {node.rework_prob} 较高")
        elif node.op_type != OpType.M and node.rework_prob > 0:
            warnings.append(f"节点 '{node.step_id}' 非测量类型但设置了返工概率")
    # 检查工作负荷
    high_load_nodes = [n for n in process.nodes if n.work_load_score >= 8]
    if len(high_load_nodes) > len(process.nodes) * 0.3:
        warnings.append(f"有 {len(high_load_nodes)} 个高负荷节点（≥8分），可能影响工人效率")
    # 计算并行组数（简化计算）
    parallel_groups = len(start_nodes)
    result = ValidationResult(
        valid=len(errors) == 0,
        errors=errors,
        warnings=warnings,
        node_count=len(process.nodes),
        edge_count=edge_count,
        parallel_groups=parallel_groups
    )
    return APIResponse(
        success=result.valid,
        message="验证通过" if result.valid else f"验证失败，存在 {len(errors)} 个错误",
        data=result.model_dump()
    )
@router.post("/save", response_model=APIResponse)
async def save_process(process: ProcessDefinition):
    """
    保存工艺流程
    将工艺流程保存到文件系统
    """
    try:
        # 生成文件名
        filename = f"{process.name.replace(' ', '_')}.json"
        data_dir = os.path.join(
```

```
            os.path.dirname(os.path.dirname(os.path.dirname(__file__))),
            "data"
        )
        os.makedirs(data_dir, exist_ok=True)
        filepath = os.path.join(data_dir, filename)
        # 保存JSON
        import json
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(process.model_dump(), f, ensure_ascii=False, indent=2)
        return APIResponse(
            success=True,
            message=f"流程已保存到 {filename}",
            data={"filepath": filepath}
        )
    except Exception as e:
        return APIResponse(
            success=False,
            message=f"保存失败: {str(e)}"
        )
@router.get("/example", response_model=APIResponse)
async def get_example_process():
    """
    获取示例工艺流程
    返回一个完整的示例工艺流程定义
    """
    example_nodes = []
    for i, row in enumerate(CSV_TEMPLATE_EXAMPLE):
        tools = [t.strip() for t in row[9].split(';') if t.strip()] if row[9] else []
        node = ProcessNode(
            step_id=row[0],
            task_name=row[1],
            op_type=OpType(row[2]),
            predecessors=row[3],
            std_duration=float(row[4]),
            time_variance=float(row[5]),
            work_load_score=int(row[6]),
            rework_prob=float(row[7]),
            required_workers=int(row[8]),
            required_tools=tools,
            x=100 + (i % 3) * 200,
            y=100 + (i // 3) * 120
        )
        example_nodes.append(node)
    process = ProcessDefinition(
        name="航空发动机装配示例流程",
        description="包含压气机转子装配、动平衡测试和整机试车的标准流程",
        nodes=example_nodes
    )
    return APIResponse(
        success=True,
```

```
        message="获取示例流程成功",
        data=process.model_dump()
    )
# 复杂示例数据 - 航空发动机完整装配流程（约35个节点，月产3台）
COMPLEX_EXAMPLE_NODES = [
    # 阶段1: 准备与来料检验
    ("S001", "风扇叶片检验", "M", "", 25, 3, 4, 0.03, 1, "检测台"),
    ("S002", "压气机叶片检验", "M", "", 30, 4, 4, 0.04, 1, "检测台"),
    ("S003", "工装准备", "H", "", 20, 2, 4, 0, 2, "装配台"),
    # 阶段2: 低压压气机装配
    ("S101", "低压转子吊装", "H", "S001;S003", 25, 4, 8, 0, 3, "吊车"),
    ("S102", "低压叶片安装", "A", "S101", 90, 10, 7, 0, 2, "装配台"),
    ("S103", "低压压气机测量", "M", "S102", 30, 4, 5, 0.08, 2, "检测台"),
    ("S104", "低压动平衡", "M", "S103", 60, 8, 6, 0.10, 2, "动平衡机"),
    # 阶段3: 高压压气机装配
    ("S201", "高压转子吊装", "H", "S002;S104", 25, 4, 8, 0, 3, "吊车"),
    ("S202", "高压叶片安装", "A", "S201", 150, 15, 7, 0, 2, "装配台"),
    ("S203", "高压压气机测量", "M", "S202", 35, 5, 5, 0.08, 2, "检测台"),
    ("S204", "高压动平衡", "M", "S203", 70, 10, 6, 0.12, 2, "动平衡机"),
    # 阶段4: 燃烧室装配
    ("S301", "燃烧室装配", "A", "S204", 120, 12, 7, 0, 2, "装配台"),
    ("S302", "燃烧室密封检查", "M", "S301", 40, 5, 5, 0.06, 2, "检测台"),
    # 阶段5: 涡轮装配
    ("S401", "涡轮装配", "A", "S302", 180, 18, 8, 0, 2, "装配台"),
    ("S402", "涡轮间隙测量", "M", "S401", 35, 5, 5, 0.08, 2, "检测台"),
    ("S403", "涡轮动平衡", "M", "S402", 75, 10, 6, 0.10, 2, "动平衡机"),
    # 阶段6: 轴系装配
    ("S501", "轴系装配", "A", "S403", 100, 10, 7, 0, 2, "装配台"),
    ("S502", "轴系测量", "M", "S501", 45, 6, 5, 0.06, 2, "检测台"),
    # 阶段7: 附件机匣装配
    ("S601", "附件机匣装配", "A", "S502", 120, 12, 6, 0, 2, "装配台"),
    ("S602", "附件系统检查", "M", "S601", 30, 4, 5, 0.05, 2, "检测台"),
    # 阶段8: 总装
    ("S701", "总装吊装", "H", "S602", 35, 5, 9, 0, 4, "吊车"),
    ("S702", "管路连接", "A", "S701", 150, 15, 6, 0, 2, ""),
    ("S703", "电气安装", "A", "S702", 80, 8, 5, 0, 2, ""),
    # 阶段9: 总装检测
    ("S801", "气密性测试", "M", "S703", 60, 8, 5, 0.05, 2, "检测台"),
    ("S802", "电气测试", "M", "S801", 45, 6, 5, 0.04, 2, "检测台"),
    # 阶段10: 试车
    ("S901", "试车准备", "H", "S802", 40, 5, 6, 0, 2, "试车台"),
    ("S902", "安装试车台", "H", "S901", 50, 6, 8, 0, 4, "试车台;吊车"),
    ("S903", "慢车试车", "M", "S902", 90, 12, 6, 0.08, 3, "试车台"),
    ("S904", "高速试车", "M", "S903", 120, 15, 7, 0.10, 3, "试车台"),
    ("S905", "数据分析", "D", "S904", 60, 8, 4, 0, 2, ""),
    ("S906", "试车后检查", "M", "S905", 45, 6, 5, 0.05, 2, "检测台"),
    # 阶段11: 最终处理
    ("S1001", "下台清洁", "H", "S906", 50, 5, 6, 0, 2, "吊车"),
    ("S1002", "文件整理", "D", "S1001", 40, 5, 3, 0, 1, ""),
    ("S1003", "质量审核", "D", "S1002", 30, 4, 3, 0, 2, ""),
```

```
    ("S1004", "包装入库", "H", "S1003", 35, 4, 6, 0, 2, ""),
]
@router.get("/example-complex", response_model=APIResponse)
async def get_complex_example_process():
    """
    获取复杂示例工艺流程
    返回航空发动机完整装配流程（约35个节点）
    设计为在人因与资源双重约束下月产约3台发动机
    总标准工时约2000分钟（约33小时），考虑：
    - 6名工人，部分任务需要2-4人
    - 关键设备约束（动平衡机2台、试车台1台等）
    - 休息规则（连续工作50分钟休息、高负荷任务后休息）
    - 返工概率（M类型任务5%-12%返工率）
    """
    nodes = []
    for i, row in enumerate(COMPLEX_EXAMPLE_NODES):
        tools = [t.strip() for t in row[9].split(';') if t.strip()] if row[9] else []
        node = ProcessNode(
            step_id=row[0],
            task_name=row[1],
            op_type=OpType(row[2]),
            predecessors=row[3],
            std_duration=float(row[4]),
            time_variance=float(row[5]),
            work_load_score=int(row[6]),
            rework_prob=float(row[7]),
            required_workers=int(row[8]),
            required_tools=tools,
            x=100 + (i % 4) * 180,
            y=80 + (i // 4) * 90
        )
        nodes.append(node)
    process = ProcessDefinition(
        name="航空发动机完整装配流程",
        description="完整的航空发动机装配流程，包含11个阶段35个工序，设计月产量约3台",
        nodes=nodes
    )
    return APIResponse(
        success=True,
        message="获取复杂示例流程成功（35个节点，预计月产3台）",
        data=process.model_dump()
    )
@router.get("/op-types", response_model=APIResponse)
async def get_op_types():
    """
    获取操作类型元数据
    返回所有操作类型的中英文名称、颜色和图标
    """
    data = []
    for op_type in OpType:
```

```
        meta = OP_TYPE_META[op_type]
        data.append({
            "value": op_type.value,
            "zh": meta["zh"],
            "en": meta["en"],
            "color": meta["color"],
            "icon": meta["icon"]
        })
    return APIResponse(
        success=True,
        message="获取操作类型成功",
        data=data
    )
@router.post("/export-csv", response_model=None)
async def export_process_csv(process: ProcessDefinition):
    """
    导出工艺流程为CSV
    """
    output = io.StringIO()
    writer = csv.writer(output)
    # 写入表头
    writer.writerow(CSV_TEMPLATE_HEADERS)
    # 写入数据
    for node in process.nodes:
        writer.writerow([
            node.step_id,
            node.task_name,
            node.op_type.value,
            node.predecessors,
            node.std_duration,
            node.time_variance,
            node.work_load_score,
            node.rework_prob,
            node.required_workers,
            ";".join(node.required_tools)
        ])
    output.seek(0)
    filename = f"{process.name.replace(' ', '_')}.csv"
    return StreamingResponse(
        io.BytesIO(output.getvalue().encode('utf-8-sig')),
        media_type="text/csv",
        headers={
            "Content-Disposition": f"attachment; filename={filename}"
        }
    )
# ===== File: app/api/results.py =====
"""
结果查询接口
提供仿真结果的查询、导出和分析功能
API端点:
```

```
- GET /api/results/{sim_id}: 获取仿真结果
- GET /api/results/list: 列出所有结果
- POST /api/results/export/gantt: 导出甘特图CSV
- POST /api/results/export/report: 导出完整报告
- GET /api/results/{sim_id}/gantt: 获取甘特图数据（支持时间筛选）
- GET /api/results/{sim_id}/kpi: 获取KPI指标
- GET /api/results/{sim_id}/bottleneck: 获取瓶颈分析
"""
import os
import io
import csv
import json
from datetime import datetime
from typing import Dict, List, Any, Optional
from fastapi import APIRouter, HTTPException, Query
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
router = APIRouter()
# ============ 数据模型 ============
class APIResponse(BaseModel):
    """统一API响应格式"""
    success: bool
    message: str
    data: Optional[Any] = None
class GanttTimeRange(BaseModel):
    """甘特图时间范围"""
    start_day: int = Field(default=1, ge=1, description="开始天数")
    start_hour: float = Field(default=0, ge=0, description="开始小时")
    end_day: int = Field(default=22, ge=1, description="结束天数")
    end_hour: float = Field(default=8, ge=0, description="结束小时")
    work_hours_per_day: int = Field(default=8, ge=1, le=24, description="每日工作小时数")
class GanttExportRequest(BaseModel):
    """甘特图导出请求"""
    sim_id: str = Field(description="仿真ID")
    time_range: Optional[GanttTimeRange] = Field(default=None, description="时间范围")
    include_rest: bool = Field(default=True, description="包含休息事件")
    include_rework: bool = Field(default=True, description="包含返工事件")
class KPIData(BaseModel):
    """KPI数据"""
    # 产量指标
    engines_completed: int = Field(description="完成发动机数量")
    target_output: int = Field(description="目标产量")
    target_achievement_rate: float = Field(description="计划达成率")
    # 时间指标
    total_sim_time: float = Field(description="总仿真时间（分钟）")
    avg_cycle_time: float = Field(description="平均单台周期时间（分钟）")
    avg_cycle_time_hours: float = Field(description="平均单台周期时间（小时）")
    # 效率指标
    avg_worker_utilization: float = Field(description="平均工人利用率")
    avg_equipment_utilization: float = Field(description="平均设备利用率")
```

```
    # 质量指标
    first_pass_rate: float = Field(description="一次通过率")
    total_reworks: int = Field(description="总返工次数")
    rework_time_ratio: float = Field(description="返工时间占比")
# ============ 模拟数据存储（与simulation.py共享） ============
# 导入simulation模块的存储
from app.api.simulation import simulation_results, SimulationResult, SimulationStatus
# ============ 辅助函数 ============
def minutes_to_day_hour(minutes: float, work_hours_per_day: int) -> Dict[str, Any]:
    """将仿真分钟转换为Day-Hour格式"""
    minutes_per_day = work_hours_per_day * 60
    day = int(minutes // minutes_per_day) + 1
    remaining_minutes = minutes % minutes_per_day
    hour = remaining_minutes / 60
    return {
        "day": day,
        "hour": round(hour, 2),
        "formatted": f"D{day} {hour:.1f}h",
        "total_minutes": minutes
    }
def day_hour_to_minutes(day: int, hour: float, work_hours_per_day: int) -> float:
    """将Day-Hour格式转换为仿真分钟"""
    minutes_per_day = work_hours_per_day * 60
    return (day - 1) * minutes_per_day + hour * 60
# ============ API端点 ============
@router.get("/{sim_id}", response_model=APIResponse)
async def get_simulation_result(sim_id: str):
    """
    获取仿真结果
    返回指定仿真ID的完整结果数据
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
    return APIResponse(
        success=True,
        message="获取结果成功",
        data=result.model_dump()
    )
@router.get("/list/all", response_model=APIResponse)
async def list_all_results(
    limit: int = Query(default=20, ge=1, le=100, description="返回数量限制"),
    status: Optional[str] = Query(default=None, description="筛选状态")
):
    """
    列出所有仿真结果
    返回仿真结果列表的概要信息
```

```
    """
    results_list = []
    for sim_id, result in simulation_results.items():
        # 状态筛选
        if status and result.status.value != status:
            continue
        results_list.append({
            "sim_id": sim_id,
            "status": result.status.value,
            "engines_completed": result.engines_completed,
            "target_achievement_rate": round(result.target_achievement_rate * 100, 1),
            "avg_cycle_time_hours": round(result.avg_cycle_time / 60, 2),
            "created_at": result.created_at,
            "completed_at": result.completed_at,
            "config_summary": {
                "num_workers": result.config.num_workers,
                "target_output": result.config.target_output,
                "work_days": result.config.work_days_per_month
            }
        })
    # 按创建时间倒序排列
    results_list.sort(key=lambda x: x["created_at"], reverse=True)
    # 限制返回数量
    results_list = results_list[:limit]
    return APIResponse(
        success=True,
        message=f"共 {len(results_list)} 条结果",
        data=results_list
    )
@router.get("/{sim_id}/gantt", response_model=APIResponse)
async def get_gantt_data(
    sim_id: str,
    start_day: int = Query(default=1, ge=1, description="开始天数"),
    start_hour: float = Query(default=0, ge=0, description="开始小时"),
    end_day: int = Query(default=22, ge=1, description="结束天数"),
    end_hour: float = Query(default=8, ge=0, description="结束小时"),
    engine_id: Optional[int] = Query(default=None, description="筛选发动机编号"),
    event_type: Optional[str] = Query(default=None, description="筛选事件类型")
):
    """
    获取甘特图数据
    支持按时间范围、发动机编号、事件类型筛选
    返回格式化的甘特图事件数据
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
```

```
    work_hours = result.config.work_hours_per_day
    # 计算时间范围（分钟）
    start_minutes = day_hour_to_minutes(start_day, start_hour, work_hours)
    end_minutes = day_hour_to_minutes(end_day, end_hour, work_hours)
    # 筛选事件
    filtered_events = []
    for event in result.gantt_events:
        # 时间范围筛选
        if event.end_time < start_minutes or event.start_time > end_minutes:
            continue
        # 发动机筛选
        if engine_id is not None and event.engine_id != engine_id:
            continue
        # 事件类型筛选
        if event_type and event.event_type != event_type:
            continue
        # 格式化时间
        event_data = event.model_dump()
        event_data["start_time_formatted"] = minutes_to_day_hour(event.start_time, work_hours)
        event_data["end_time_formatted"] = minutes_to_day_hour(event.end_time, work_hours)
        event_data["duration_minutes"] = event.end_time - event.start_time
        filtered_events.append(event_data)
    # 按发动机和开始时间排序
    filtered_events.sort(key=lambda x: (x["engine_id"], x["start_time"]))
    return APIResponse(
        success=True,
        message=f"获取 {len(filtered_events)} 个甘特图事件",
        data={
            "events": filtered_events,
            "time_range": {
                "start": minutes_to_day_hour(start_minutes, work_hours),
                "end": minutes_to_day_hour(end_minutes, work_hours)
            },
            "total_events": len(result.gantt_events),
            "filtered_events": len(filtered_events)
        }
    )
@router.get("/{sim_id}/kpi", response_model=APIResponse)
async def get_kpi_data(sim_id: str):
    """
    获取KPI指标
    返回仿真结果的关键性能指标
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
    # 计算工人平均利用率
```

```
    avg_worker_util = 0
    if result.worker_stats:
        avg_worker_util = sum(w.utilization_rate for w in result.worker_stats) / len(result.worker_stats)
    # 计算设备平均利用率
    avg_equip_util = 0
    if result.equipment_stats:
        avg_equip_util = sum(e.utilization_rate for e in result.equipment_stats) / len(result.equipment_stats)
    # 计算返工时间占比
    total_time = sum(e.end_time - e.start_time for e in result.gantt_events)
    rework_time = sum(e.end_time - e.start_time for e in result.gantt_events if e.event_type == "REWORK")
    rework_ratio = rework_time / total_time if total_time > 0 else 0
    kpi = KPIData(
        engines_completed=result.engines_completed,
        target_output=result.config.target_output,
        target_achievement_rate=result.target_achievement_rate,
        total_sim_time=result.sim_duration,
        avg_cycle_time=result.avg_cycle_time,
        avg_cycle_time_hours=result.avg_cycle_time / 60,
        avg_worker_utilization=avg_worker_util,
        avg_equipment_utilization=avg_equip_util,
        first_pass_rate=result.quality_stats.first_pass_rate,
        total_reworks=result.quality_stats.total_reworks,
        rework_time_ratio=rework_ratio
    )
    return APIResponse(
        success=True,
        message="获取KPI成功",
        data=kpi.model_dump()
    )
@router.get("/{sim_id}/bottleneck", response_model=APIResponse)
async def get_bottleneck_analysis(sim_id: str):
    """
    获取瓶颈分析
    分析生产过程中的瓶颈，包括：
    - 设备瓶颈（高利用率设备）
    - 工人瓶颈（高负荷工人）
    - 等待时间瓶颈（长等待任务）
    - 返工瓶颈（高返工率任务）
    返回瓶颈列表、汇总和改进建议
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
    # 使用统计模块的瓶颈分析
    from app.utils.statistics import analyze_bottlenecks
    analysis = analyze_bottlenecks(result)
    return APIResponse(
```

```
        success=True,
        message=f"识别到 {len(analysis.bottlenecks)} 个瓶颈",
        data=analysis.to_dict()
    )
@router.post("/export/gantt")
async def export_gantt_csv(request: GanttExportRequest):
    """
    导出甘特图CSV
    将甘特图数据导出为CSV文件
    """
    if request.sim_id not in simulation_results:
        raise HTTPException(status_code=404, detail=f"仿真结果 {request.sim_id} 不存在")
    result = simulation_results[request.sim_id]
    work_hours = result.config.work_hours_per_day
    # 筛选事件
    events = result.gantt_events
    if request.time_range:
        start_minutes = day_hour_to_minutes(
            request.time_range.start_day,
            request.time_range.start_hour,
            work_hours
        )
        end_minutes = day_hour_to_minutes(
            request.time_range.end_day,
            request.time_range.end_hour,
            work_hours
        )
        events = [e for e in events if e.start_time >= start_minutes and e.end_time <= end_minutes]
    if not request.include_rest:
        events = [e for e in events if e.event_type != "REST"]
    if not request.include_rework:
        events = [e for e in events if e.event_type != "REWORK"]
    # 生成CSV
    output = io.StringIO()
    writer = csv.writer(output)
    # 表头
    headers = [
        "engine_id", "step_id", "task_name", "op_type",
        "start_day", "start_hour", "end_day", "end_hour",
        "duration_minutes", "event_type", "workers", "equipment", "rework_count"
    ]
    writer.writerow(headers)
    # 数据行
    for event in events:
        start_time = minutes_to_day_hour(event.start_time, work_hours)
        end_time = minutes_to_day_hour(event.end_time, work_hours)
        writer.writerow([
            event.engine_id,
            event.step_id,
            event.task_name,
```

```
            event.op_type,
            start_time["day"],
            start_time["hour"],
            end_time["day"],
            end_time["hour"],
            round(event.end_time - event.start_time, 2),
            event.event_type,
            ";".join(event.worker_ids),
            ";".join(event.equipment_used),
            event.rework_count
        ])
    output.seek(0)
    filename = f"gantt_{request.sim_id[:8]}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    return StreamingResponse(
        io.BytesIO(output.getvalue().encode('utf-8-sig')),
        media_type="text/csv",
        headers={
            "Content-Disposition": f"attachment; filename={filename}"
        }
    )
@router.post("/export/report")
async def export_report(sim_id: str):
    """
    导出完整报告
    生成包含配置、KPI、甘特图数据的JSON报告
    """
    if sim_id not in simulation_results:
        raise HTTPException(status_code=404, detail=f"仿真结果 {sim_id} 不存在")
    result = simulation_results[sim_id]
    work_hours = result.config.work_hours_per_day
    # 获取瓶颈分析
    from app.utils.statistics import analyze_bottlenecks
    bottleneck_analysis = analyze_bottlenecks(result)
    # 构建报告
    report = {
        "report_info": {
            "sim_id": sim_id,
            "generated_at": datetime.now().isoformat(),
            "system": "航空发动机装配排产仿真系统",
            "version": "1.0.0"
        },
        "config": result.config.model_dump(),
        "summary": {
            "status": result.status.value,
            "engines_completed": result.engines_completed,
            "target_achievement_rate": f"{result.target_achievement_rate * 100:.1f}%",
            "avg_cycle_time": f"{result.avg_cycle_time / 60:.2f} 小时",
            "total_sim_time": f"{result.sim_duration / 60:.2f} 小时"
        },
        "quality_stats": result.quality_stats.model_dump(),
```

```
        "worker_stats": [w.model_dump() for w in result.worker_stats],
        "equipment_stats": [e.model_dump() for e in result.equipment_stats],
        "bottleneck_analysis": bottleneck_analysis.to_dict(),
        "gantt_events_count": len(result.gantt_events)
    }
    # 生成JSON
    output = json.dumps(report, ensure_ascii=False, indent=2)
    filename = f"report_{sim_id[:8]}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    return StreamingResponse(
        io.BytesIO(output.encode('utf-8')),
        media_type="application/json",
        headers={
            "Content-Disposition": f"attachment; filename={filename}"
        }
    )
@router.get("/{sim_id}/worker-stats", response_model=APIResponse)
async def get_worker_stats(sim_id: str):
    """
    获取工人统计数据
    返回各工人的利用率、工作时间、休息时间等详细数据
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
    # 格式化工人统计
    worker_data = []
    for stat in result.worker_stats:
        worker_data.append({
            "worker_id": stat.resource_id,
            "work_time_hours": round(stat.work_time / 60, 2),
            "rest_time_hours": round(stat.rest_time / 60, 2),
            "idle_time_hours": round(stat.idle_time / 60, 2),
            "utilization_rate": round(stat.utilization_rate * 100, 1),
            "tasks_completed": stat.tasks_completed
        })
    return APIResponse(
        success=True,
        message="获取工人统计成功",
        data=worker_data
    )
@router.get("/{sim_id}/equipment-stats", response_model=APIResponse)
async def get_equipment_stats(sim_id: str):
    """
    获取设备统计数据
    返回各设备的利用率详细数据，包括关键设备和无限制设备
    """
    if sim_id not in simulation_results:
```

```
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
    result = simulation_results[sim_id]
    # 格式化设备统计
    equipment_data = []
    critical_equipment = []
    unlimited_equipment = []
    for stat in result.equipment_stats:
        item = {
            "equipment_name": stat.resource_id,
            "work_time_hours": round(stat.work_time / 60, 2),
            "idle_time_hours": round(stat.idle_time / 60, 2),
            "utilization_rate": round(stat.utilization_rate * 100, 1),
            "tasks_served": stat.tasks_completed,
            "is_unlimited": getattr(stat, 'is_unlimited', False),
            "is_bottleneck": stat.utilization_rate > 0.8
        }
        equipment_data.append(item)
        if getattr(stat, 'is_unlimited', False):
            unlimited_equipment.append(item)
        else:
            critical_equipment.append(item)
    return APIResponse(
        success=True,
        message="获取设备统计成功",
        data={
            "all": equipment_data,
            "critical": critical_equipment,
            "unlimited": unlimited_equipment,
            "summary": {
                "total_count": len(equipment_data),
                "critical_count": len(critical_equipment),
                "unlimited_count": len(unlimited_equipment),
                "bottleneck_count": len([e for e in equipment_data if e["is_bottleneck"]])
            }
        }
    )
@router.delete("/{sim_id}", response_model=APIResponse)
async def delete_result(sim_id: str):
    """
    删除仿真结果
    从存储中删除指定的仿真结果
    """
    if sim_id not in simulation_results:
        return APIResponse(
            success=False,
            message=f"仿真结果 {sim_id} 不存在"
        )
```

```
    del simulation_results[sim_id]
    return APIResponse(
        success=True,
        message=f"仿真结果 {sim_id} 已删除"
    )
# ===== File: app/api/simulation.py =====
"""
仿真控制接口
提供仿真的启动、停止、状态查询等功能
API端点:
- POST /api/simulation/run: 运行仿真
- GET /api/simulation/test: 运行测试仿真
- GET /api/simulation/status/{sim_id}: 获取仿真状态
- POST /api/simulation/stop/{sim_id}: 停止仿真
"""
import uuid
import time
from datetime import datetime
from typing import Dict, List, Any, Optional
from enum import Enum
from fastapi import APIRouter, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field
router = APIRouter()
# ============ 枚举和常量 ============
class SimulationStatus(str, Enum):
    """仿真状态枚举"""
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"
class OpType(str, Enum):
    """操作类型枚举"""
    H = "H"
    A = "A"
    M = "M"
    T = "T"
    D = "D"
# ============ 数据模型 ============
from app.models.config_model import GlobalConfig
class ProcessNode(BaseModel):
    """工艺节点模型"""
    step_id: str
    task_name: str
    op_type: OpType
    predecessors: str = ""
    std_duration: float
    time_variance: float = 0.0
    work_load_score: int = Field(default=5, ge=1, le=10)
    rework_prob: float = Field(default=0.0, ge=0, le=1)
```

```
    required_workers: int = Field(default=1, ge=1)
    required_tools: List[str] = []
    station: str = ""  # 工位ID
    x: float = 0
    y: float = 0
class ProcessDefinition(BaseModel):
    """工艺流程定义"""
    name: str = "未命名流程"
    description: str = ""
    nodes: List[ProcessNode] = []
class SimulationRequest(BaseModel):
    """仿真请求"""
    config: GlobalConfig
    process: ProcessDefinition
class GanttEvent(BaseModel):
    """甘特图事件"""
    engine_id: int = Field(description="发动机编号")
    step_id: str = Field(description="步骤ID")
    task_name: str = Field(description="任务名称")
    op_type: str = Field(description="操作类型")
    start_time: float = Field(description="开始时间（分钟）")
    end_time: float = Field(description="结束时间（分钟）")
    event_type: str = Field(description="事件类型（NORMAL/REST/REWORK/WAITING）")
    worker_ids: List[str] = Field(default=[], description="执行工人列表")
    equipment_used: List[str] = Field(default=[], description="使用的关键设备")
    rework_count: int = Field(default=0, description="返工次数")
class ResourceUtilization(BaseModel):
    """资源利用率统计"""
    resource_id: str = Field(description="资源ID")
    resource_type: str = Field(description="资源类型")
    total_time: float = Field(description="总时间")
    work_time: float = Field(description="工作时间")
    rest_time: float = Field(default=0, description="休息时间")
    idle_time: float = Field(default=0, description="空闲时间")
    utilization_rate: float = Field(description="利用率")
    tasks_completed: int = Field(default=0, description="完成任务数")
class QualityStats(BaseModel):
    """质量统计"""
    total_inspections: int = Field(default=0, description="总检测次数")
    total_reworks: int = Field(default=0, description="总返工次数")
    first_pass_rate: float = Field(default=1.0, description="一次通过率")
    rework_time_total: float = Field(default=0, description="返工总耗时")
class SimulationResult(BaseModel):
    """仿真结果"""
    sim_id: str = Field(description="仿真ID")
    status: SimulationStatus = Field(description="仿真状态")
    config: GlobalConfig = Field(description="使用的配置")
    sim_duration: float = Field(description="实际仿真时长（分钟）")
    engines_completed: int = Field(description="完成发动机数量")
    target_achievement_rate: float = Field(description="计划达成率")
```

```
    avg_cycle_time: float = Field(description="平均单台周期时间（分钟）")
    worker_stats: List[ResourceUtilization] = Field(default=[], description="工人统计")
    equipment_stats: List[ResourceUtilization] = Field(default=[], description="设备统计")
    quality_stats: QualityStats = Field(description="质量统计")
    gantt_events: List[GanttEvent] = Field(default=[], description="甘特图事件列表")
    time_mapping: Dict[str, Any] = Field(default={}, description="时间映射元数据")
    created_at: str = Field(description="创建时间")
    completed_at: Optional[str] = Field(default=None, description="完成时间")
class APIResponse(BaseModel):
    """统一API响应格式"""
    success: bool
    message: str
    data: Optional[Any] = None
# ============ 仿真结果存储 ============
# 内存存储（生产环境应使用数据库）
simulation_results: Dict[str, SimulationResult] = {}
running_simulations: Dict[str, bool] = {}
# ============ 辅助函数 ============
def to_calendar_time(minutes: float, work_hours_per_day: int) -> str:
    """将仿真分钟转换为 Day-Hour 格式"""
    minutes_per_day = work_hours_per_day * 60
    day = int(minutes // minutes_per_day) + 1
    hour_in_day = (minutes % minutes_per_day) / 60
    return f"D{day} {hour_in_day:.1f}h"
def generate_mock_simulation_result(
    sim_id: str,
    config: GlobalConfig,
    process: ProcessDefinition
) -> SimulationResult:
    """
    生成模拟仿真结果（用于测试）
    实际应用中应使用SimPy仿真引擎
    """
    import random
    # 计算基本参数
    total_std_time = sum(node.std_duration for node in process.nodes)
    sim_time = config.sim_time_minutes
    # 估算产能
    estimated_engines = max(1, int(sim_time / (total_std_time * 1.2)))
    engines_completed = min(estimated_engines, config.target_output + 1)
    # 生成甘特图事件
    gantt_events = []
    current_time = 0
    for engine_id in range(1, engines_completed + 1):
        completed_nodes = set()
        node_map = {n.step_id: n for n in process.nodes}
        while len(completed_nodes) < len(process.nodes):
            # 找到可执行的节点
            for node in process.nodes:
                if node.step_id in completed_nodes:
```

```
        if (e.key === 'Enter') {
            handleConfirm();
        }
    });
    input.addEventListener('keydown', (e) => {
        if (e.key === 'Escape') {
            handleCancel();
        }
    });
}
// 滚轮缩放
function handleCanvasWheel(e) {
    e.preventDefault();
    const delta = e.deltaY > 0 ? 0.9 : 1.1;
    scale = Math.min(Math.max(scale * delta, 0.3), 3);
    renderCanvas();
}
// 拖放事件
function handleCanvasDrop(e) {
    e.preventDefault();
    const opType = e.dataTransfer.getData('opType');
    if (!opType) return;
    const pos = getMousePos(e);
    const newId = `node_${Date.now()}`;
    const stepId = `S${String(state.nodes.length + 1).padStart(3, '0')}`;
    state.nodes.push({
        id: newId,
        stepId: stepId,
        taskName: '新节点',
        opType: opType,
        predecessors: '',
        stdDuration: 30,
        timeVariance: 5,
        workLoadScore: 5,
        reworkProb: 0,
        requiredWorkers: 1,
        requiredTools: '',
        station: state.stationFilter || 'ST01',
        x: pos.x - 60,
        y: pos.y - 25
    });
    renderCanvas();
    showToast('节点已创建，双击编辑', 'success');
}
// 键盘事件
function handleKeyDown(e) {
    if (e.key === 'Delete' && state.selectedNode) {
        deleteSelectedNode();
    }
}
```

```
/* ===== File: frontend/js/charts.js ===== */
/**
 * 图表模块 - 疲劳图表、甘特图
 */
// 全局主题优化：统一文字与边框颜色
if (window.Chart) {
    Chart.defaults.color = '#9ca3af';
    Chart.defaults.borderColor = '#374151';
}
function updateFatigueChart() {
    if (!state.simulationResult) return;
    const selectedWorker = document.getElementById('fatigueWorkerSelect').value;
    const fatigueData = state.simulationResult.worker_fatigue_data || state.simulationResult.worker_stats || [];
    const colors = ['#3b82f6', '#10b981', '#f59e0b', '#ef4444', '#8b5cf6', '#ec4899', '#06b6d4', '#84cc16'];
    let datasets = [];
    if (selectedWorker === 'all') {
        datasets = fatigueData.map((worker, idx) => {
            const workerId = worker.worker_id || worker.resource_id;
            const history = worker.fatigue_history || [];
            if (history.length > 0) {
                return {
                    label: workerId,
                    data: history.map(h => ({ x: h[0], y: h[1] })),
                    borderColor: colors[idx % colors.length],
                    backgroundColor: 'transparent',
                    tension: 0.3,
                    pointRadius: 0
                };
            } else {
                return {
                    label: workerId,
                    data: [{ x: 0, y: 0 }, { x: state.simulationResult.sim_duration, y: worker.fatigue_level || 0 }],
                    borderColor: colors[idx % colors.length],
                    backgroundColor: 'transparent',
                    tension: 0,
                    pointRadius: 2
                };
            }
        });
    } else {
        const worker = fatigueData.find(w => (w.worker_id || w.resource_id) === selectedWorker);
        if (worker) {
            const history = worker.fatigue_history || [];
            if (history.length > 0) {
                datasets = [{
                    label: selectedWorker,
                    data: history.map(h => ({ x: h[0], y: h[1] })),
                    borderColor: '#3b82f6',
                    backgroundColor: 'rgba(59, 130, 246, 0.1)',
                    fill: true,
```

```
                    tension: 0.3,
                    pointRadius: 2
                }];
            } else {
                datasets = [{
                    label: selectedWorker,
                    data: [{ x: 0, y: 0 }, { x: state.simulationResult.sim_duration, y: worker.fatigue_level || 0 }],
                    borderColor: '#3b82f6',
                    backgroundColor: 'rgba(59, 130, 246, 0.1)',
                    fill: true,
                    tension: 0,
                    pointRadius: 4
                }];
            }
        }
    }
    if (fatigueChart) fatigueChart.destroy();
    fatigueChart = new Chart(document.getElementById('fatigueChart'), {
        type: 'line',
        data: { datasets },
        options: {
            responsive: true,
            scales: {
                x: {
                    type: 'linear',
                    title: { display: true, text: '时间（分钟）', color: '#9ca3af' },
                    ticks: { color: '#9ca3af' },
                    grid: { display: false }
                },
                y: {
                    min: 0,
                    max: 100,
                    title: { display: true, text: '疲劳度', color: '#9ca3af' },
                    ticks: { color: '#9ca3af' },
                    grid: { display: false }
                }
            },
            plugins: {
                legend: {
                    display: selectedWorker === 'all',
                    labels: { color: '#9ca3af' }
                }
            }
        }
    });
}
// ============================================================
// Gantt Chart - 优化版：自适应宽度 + 增大行高
// ============================================================
function initGanttControls(result) {
```

```
    const config = getConfigFromUI();
    const totalDays = config.work_days_per_month;
    const hoursPerDay = config.work_hours_per_day;
    const startDaySelect = document.getElementById('ganttStartDay');
    const endDaySelect = document.getElementById('ganttEndDay');
    const startHourSelect = document.getElementById('ganttStartHour');
    const endHourSelect = document.getElementById('ganttEndHour');
    startDaySelect.innerHTML = '';
    endDaySelect.innerHTML = '';
    for (let d = 1; d <= totalDays; d++) {
        startDaySelect.innerHTML += `<option value="${d}">第${d}天</option>`;
        endDaySelect.innerHTML += `<option value="${d}">第${d}天</option>`;
    }
    const actualDays = Math.ceil(result.sim_duration / (hoursPerDay * 60));
    endDaySelect.value = Math.min(Math.max(actualDays, 3), totalDays);
    startHourSelect.innerHTML = '';
    endHourSelect.innerHTML = '';
    for (let h = 0; h < hoursPerDay; h++) {
        startHourSelect.innerHTML += `<option value="${h}">${h}:00</option>`;
        endHourSelect.innerHTML += `<option value="${h+1}">${h+1}:00</option>`;
    }
    endHourSelect.value = hoursPerDay;
    updateGantt();
}
function updateGantt() {
    if (!state.simulationResult) {
        showToast('请先运行仿真', 'warning');
        return;
    }
    const config = getConfigFromUI();
    const hoursPerDay = config.work_hours_per_day;
    const minutesPerDay = hoursPerDay * 60;
    const startDay = parseInt(document.getElementById('ganttStartDay').value);
    const startHour = parseInt(document.getElementById('ganttStartHour').value);
    const endDay = parseInt(document.getElementById('ganttEndDay').value);
    const endHour = parseInt(document.getElementById('ganttEndHour').value);
    const startMinute = (startDay - 1) * minutesPerDay + startHour * 60;
    const endMinute = (endDay - 1) * minutesPerDay + endHour * 60;
    const totalMinutes = endMinute - startMinute;
    // ===== 计算hourWidth =====
    const engineRight = document.getElementById('engineGanttRight');
    const availableWidth = engineRight ? (engineRight.clientWidth - 20) : 800;
    const totalHours = totalMinutes / 60;
    // 最小30px保证可读性，无上限让其自适应
    const hourWidth = Math.max(30, availableWidth / totalHours);
    renderTimeline('ganttTimeline', startDay, startHour, endDay, endHour, hoursPerDay, hourWidth);
    renderTimeline('workerGanttTimeline', startDay, startHour, endDay, endHour, hoursPerDay, hourWidth);
    renderEngineGantt(startMinute, endMinute, hoursPerDay, hourWidth);
    renderWorkerGantt(startMinute, endMinute, hoursPerDay, hourWidth);
    // 设置滚动同步
```

```
    setupScrollSync();
}
// 设置左右、上下滚动同步
function setupScrollSync() {
    // 发动机甘特图滚动同步
    const engineLabels = document.getElementById('engineGanttLabels');
    const engineBody = document.getElementById('ganttBody');
    if (engineLabels && engineBody) {
        engineBody.onscroll = function() {
            engineLabels.scrollTop = engineBody.scrollTop;
        };
        engineLabels.onscroll = function() {
            engineBody.scrollTop = engineLabels.scrollTop;
        };
    }
    // 工人甘特图滚动同步
    const workerLabels = document.getElementById('workerGanttLabels');
    const workerBody = document.getElementById('workerGanttBody');
    if (workerLabels && workerBody) {
        workerBody.onscroll = function() {
            workerLabels.scrollTop = workerBody.scrollTop;
        };
        workerLabels.onscroll = function() {
            workerBody.scrollTop = workerLabels.scrollTop;
        };
    }
}
function renderTimeline(containerId, startDay, startHour, endDay, endHour, hoursPerDay, hourWidth) {
    const timeline = document.getElementById(containerId);
    timeline.innerHTML = '';
    for (let d = startDay; d <= endDay; d++) {
        const dayStartHour = d === startDay ? startHour : 0;
        const dayEndHour = d === endDay ? endHour : hoursPerDay;
        for (let h = dayStartHour; h < dayEndHour; h++) {
            const cell = document.createElement('div');
            cell.className = 'gantt-time-cell' + (h === 0 ? ' day-start' : '');
            cell.style.width = hourWidth + 'px';
            cell.style.minWidth = hourWidth + 'px';
            cell.textContent = `D${d} ${h}h`;
            timeline.appendChild(cell);
        }
    }
}
function renderEngineGantt(startMinute, endMinute, hoursPerDay, hourWidth) {
    const events = state.simulationResult.gantt_events.filter(e =>
        e.end_time > startMinute && e.start_time < endMinute
    );
    const totalWidth = (endMinute - startMinute) / 60 * hourWidth;
    const engineGroups = {};
    events.forEach(e => {
```

```
        const key = `Engine ${e.engine_id}`;
        if (!engineGroups[key]) engineGroups[key] = [];
        engineGroups[key].push(e);
    });
    const labels = document.getElementById('engineGanttLabels');
    const body = document.getElementById('ganttBody');
    labels.innerHTML = '';
    body.innerHTML = '';
    const sortedEngines = Object.entries(engineGroups).sort((a, b) => {
        return parseInt(a[0].split(' ')[1]) - parseInt(b[0].split(' ')[1]);
    });
    sortedEngines.forEach(([label, evts]) => {
        // 左侧标签
        const labelRow = document.createElement('div');
        labelRow.className = 'gantt-left-row';
        labelRow.textContent = label;
        labels.appendChild(labelRow);
        // 右侧内容
        const row = document.createElement('div');
        row.className = 'gantt-row';
        const rowContent = document.createElement('div');
        rowContent.className = 'gantt-row-content';
        rowContent.style.width = totalWidth + 'px';
        evts.forEach(evt => {
            const bar = createGanttBar(evt, startMinute, endMinute, hourWidth);
            rowContent.appendChild(bar);
        });
        row.appendChild(rowContent);
        body.appendChild(row);
    });
}
function renderWorkerGantt(startMinute, endMinute, hoursPerDay, hourWidth) {
    const events = state.simulationResult.gantt_events.filter(e =>
        e.end_time > startMinute && e.start_time < endMinute
    );
    const totalWidth = (endMinute - startMinute) / 60 * hourWidth;
    const workerGroups = {};
    events.forEach(e => {
        if (e.worker_ids && e.worker_ids.length > 0) {
            e.worker_ids.forEach(workerId => {
                if (!workerGroups[workerId]) workerGroups[workerId] = [];
                workerGroups[workerId].push({ ...e, displayWorker: workerId });
            });
        }
    });
    const labels = document.getElementById('workerGanttLabels');
    const body = document.getElementById('workerGanttBody');
    labels.innerHTML = '';
    body.innerHTML = '';
    const sortedWorkers = Object.keys(workerGroups).sort((a, b) => {
```

```
        return parseInt(a.replace(/\D/g, '')) - parseInt(b.replace(/\D/g, ''));
    });
    const config = getConfigFromUI();
    // 先渲染有任务的工人
    sortedWorkers.forEach(workerId => {
        // 左侧标签
        const labelRow = document.createElement('div');
        labelRow.className = 'gantt-left-row';
        labelRow.textContent = workerId;
        labels.appendChild(labelRow);
        // 右侧内容
        const row = document.createElement('div');
        row.className = 'gantt-row';
        const rowContent = document.createElement('div');
        rowContent.className = 'gantt-row-content';
        rowContent.style.width = totalWidth + 'px';
        workerGroups[workerId].forEach(evt => {
            const bar = createGanttBar(evt, startMinute, endMinute, hourWidth, true);
            rowContent.appendChild(bar);
        });
        row.appendChild(rowContent);
        body.appendChild(row);
    });
    // 渲染没有任务的工人
    for (let i = 1; i <= config.num_workers; i++) {
        const workerId = `Worker_${String(i).padStart(2, '0')}`;
        if (!workerGroups[workerId]) {
            // 左侧标签
            const labelRow = document.createElement('div');
            labelRow.className = 'gantt-left-row';
            labelRow.textContent = workerId;
            labelRow.style.color = '#6b7280';
            labels.appendChild(labelRow);
            // 右侧内容
            const row = document.createElement('div');
            row.className = 'gantt-row';
            const rowContent = document.createElement('div');
            rowContent.className = 'gantt-row-content';
            rowContent.style.width = totalWidth + 'px';
            rowContent.innerHTML = '<span style="color: #6b7280; font-size: 0.75rem; padding-left: 10px;">（本时段无任务）</span>';
            row.appendChild(rowContent);
            body.appendChild(row);
        }
    }
}
function createWorkerGanttRow(workerId, events, startMinute, endMinute, hourWidth, totalWidth) {
    // 此函数已废弃，逻辑已合并到renderWorkerGantt中
    return null;
}
function createGanttBar(evt, startMinute, endMinute, hourWidth, isWorkerView = false) {
```

```
    const bar = document.createElement('div');
    const eventType = evt.event_type.toLowerCase();
    bar.className = 'gantt-bar ' + eventType;
    const evtStart = Math.max(evt.start_time, startMinute);
    const evtEnd = Math.min(evt.end_time, endMinute);
    const left = (evtStart - startMinute) / 60 * hourWidth;
    const width = Math.max((evtEnd - evtStart) / 60 * hourWidth, 6);
    bar.style.left = left + 'px';
    bar.style.width = width + 'px';
    const duration = (evt.end_time - evt.start_time).toFixed(0);
    let tooltipText = `${evt.task_name}\n时间: ${evt.start_time.toFixed(0)}-${evt.end_time.toFixed(0)}分钟 (${duration}分钟)\n类型: ${evt.e
    vent_type}`;
    if (evt.rework_count > 0) tooltipText += `\n返工次数: ${evt.rework_count}`;
    if (!isWorkerView && evt.worker_ids && evt.worker_ids.length > 0) {
        tooltipText += `\n工人: ${evt.worker_ids.join(', ')}`;
    }
    if (isWorkerView) tooltipText += `\n发动机: Engine ${evt.engine_id}`;
    bar.title = tooltipText;
    // 根据宽度决定显示内容
    if (width > 50) {
        bar.textContent = isWorkerView ? `E${evt.engine_id}` : evt.step_id;
    } else if (width > 30) {
        bar.textContent = isWorkerView ? evt.engine_id : evt.step_id.slice(-3);
    }
    return bar;
}
function exportGanttCSV() {
    if (!state.simulationResult) {
        showToast('请先运行仿真', 'warning');
        return;
    }
    let csv = 'engine_id,step_id,task_name,event_type,start_time,end_time,duration,workers\n';
    state.simulationResult.gantt_events.forEach(e => {
        csv += `${e.engine_id},${e.step_id},"${e.task_name}",${e.event_type},${e.start_time},${e.end_time},${e.end_time - e.start_time},"${
    (e.worker_ids || []).join(';')}"\n`;
    });
    const blob = new Blob([csv], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'gantt_chart.csv';
    a.click();
    URL.revokeObjectURL(url);
    showToast('甘特图CSV已导出', 'success');
}
/* ===== File: frontend/js/config.js ===== */
/**
 * 配置模块 - 常量和默认配置
 */
// 默认工位配置
```

```
const DEFAULT_STATIONS = [
    { id: 'ST01', name: '原材料入库区', color: '#3b82f6', order: 1 },
    { id: 'ST02', name: '预处理工区', color: '#8b5cf6', order: 2 },
    { id: 'ST03', name: '压气机模块装配区', color: '#10b981', order: 3 },
    { id: 'ST04', name: '燃烧室模块装配区', color: '#f59e0b', order: 4 },
    { id: 'ST05', name: '涡轮模块装配区', color: '#ef4444', order: 5 },
    { id: 'ST06', name: '齿轮箱模块装配区', color: '#06b6d4', order: 6 },
    { id: 'ST07', name: '燃油系统模块装配区', color: '#ec4899', order: 7 },
    { id: 'ST08', name: '点火系统模块装配区', color: '#84cc16', order: 8 },
    { id: 'ST09', name: '润滑系统模块装配区', color: '#a855f7', order: 9 },
    { id: 'ST10', name: '进气系统模块装配区', color: '#14b8a6', order: 10 },
    { id: 'ST11', name: '总装集成工区', color: '#f97316', order: 11 },
    { id: 'ST12', name: '整机测试工区', color: '#6366f1', order: 12 },
    { id: 'ST13', name: '包装发运区', color: '#22c55e', order: 13 },
    { id: 'ST14', name: '返修工区', color: '#dc2626', order: 14 }
];
// 操作类型颜色
const OP_TYPE_COLORS = {
    'H': '#3b82f6',  // 取/放 - 蓝色
    'A': '#10b981',  // 装配 - 绿色
    'M': '#f59e0b',  // 测量 - 橙色
    'T': '#8b5cf6',  // 工具操作 - 紫色
    'D': '#6b7280'   // 数据记录 - 灰色
};
// 操作类型图标
const OP_TYPE_ICONS = {
    'H': '  ',
    'A': '  ',
    'M': '  ',
    'T': '    ',
    'D': '  '
};
// 工位容器状态（位置、大小、折叠状态）
const stationContainers = {};
// 默认配置
const DEFAULT_CONFIG = {
    workHoursPerDay: 8,
    workDaysPerMonth: 22,
    numWorkers: 8,
    targetOutput: 3,
    criticalEquipment: {
        '动平衡机': 2,
        '检测台': 2,
        '叶片安装夹具': 3,
        '液压拉伸器': 2,
        '感应加热器': 2,
        '起重设备': 1
    },
    restTimeThreshold: 50,
    restDurationTime: 5,
```

```
    restLoadThreshold: 7,
    restDurationLoad: 3,
    pipelineMode: true,
    stationConstraintMode: false
};
/* ===== File: frontend/js/csv.js ===== */
/**
 * CSV导入导出和数据处理模块
 */
// 导入CSV
function importCSV() {
    document.getElementById('csvFileInput').click();
}
// 处理CSV导入
function handleCSVImport(event) {
    const file = event.target.files[0];
    if (!file) return;
    const formData = new FormData();
    formData.append('file', file);
    fetch('/api/process/parse-csv', {
        method: 'POST',
        body: formData
    })
    .then(res => res.json())
    .then(data => {
        if (data.success) {
            loadProcessData(data.data);
            showToast('CSV导入成功', 'success');
        } else {
            showToast(data.message || '导入失败', 'error');
        }
    })
    .catch(err => {
        showToast('导入失败', 'error');
    });
    event.target.value = '';
}
// 导出CSV
function exportCSV() {
    const process = buildProcessDefinition();
    fetch('/api/process/export-csv', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(process)
    })
    .then(res => res.blob())
    .then(blob => {
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
```

```
        a.download = 'process.csv';
        a.click();
        URL.revokeObjectURL(url);
        showToast('CSV已导出', 'success');
    })
    .catch(err => {
        showToast('导出失败', 'error');
    });
}
// 构建流程定义
function buildProcessDefinition() {
    return {
        name: '工艺流程',
        description: '',
        nodes: state.nodes.map(n => ({
            step_id: n.stepId,
            task_name: n.taskName,
            op_type: n.opType,
            predecessors: n.predecessors,
            std_duration: n.stdDuration,
            time_variance: n.timeVariance,
            work_load_score: n.workLoadScore,
            rework_prob: n.reworkProb,
            required_workers: n.requiredWorkers,
            required_tools: n.requiredTools ? n.requiredTools.split(';').map(t => t.trim()).filter(t => t) : [],
            station: n.station || 'ST01',
            x: n.x,
            y: n.y
        }))
    };
}
// 获取UI配置
function getConfigFromUI() {
    return {
        work_hours_per_day: parseInt(document.getElementById('workHoursPerDay').value),
        work_days_per_month: parseInt(document.getElementById('workDaysPerMonth').value),
        num_workers: parseInt(document.getElementById('numWorkers').value),
        target_output: parseInt(document.getElementById('targetOutput').value),
        critical_equipment: state.config.criticalEquipment,
        rest_time_threshold: parseInt(document.getElementById('restTimeThreshold').value),
        rest_duration_time: parseInt(document.getElementById('restDurationTime').value),
        rest_load_threshold: parseInt(document.getElementById('restLoadThreshold').value),
        rest_duration_load: parseInt(document.getElementById('restDurationLoad').value),
        pipeline_mode: document.getElementById('pipelineMode').checked,
        station_constraint_mode: document.getElementById('stationConstraintMode').checked
    };
}
/* ===== File: frontend/js/data.js ===== */
/**
 * 示例数据模块 - 航空发动机整机装配流程（基础案例，源自 data/3.csv）
```

```
 * 共 79 个工序节点，含 P/D/G/F 四条并行模块装配线与 B 系列预装，最终汇入总装 A 与试车 Q。
 */
const COMPLEX_PROCESS_NODES = [
    { step_id: 'S001', task_name: '零件接收与登记', op_type: 'A', predecessors: '', std_duration: 25, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST01' },
    { step_id: 'S002', task_name: '超声波清洗', op_type: 'A', predecessors: 'S001', std_duration: 25, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST01' },
    { step_id: 'S003', task_name: '高压清洗', op_type: 'A', predecessors: 'S002', std_duration: 25, time_variance: 5, work_load_score: 5, r
    ework_prob: 0, required_workers: 1, required_tools: '', station: 'ST01' },
    { step_id: 'S004', task_name: '检验包装', op_type: 'A', predecessors: 'S003', std_duration: 25, time_variance: 5, work_load_score: 5, r
    ework_prob: 0, required_workers: 1, required_tools: '', station: 'ST01' },
    { step_id: 'P001', task_name: '转子叶片安装', op_type: 'A', predecessors: 'S004', std_duration: 60, time_variance: 5, work_load_score:
    5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P002', task_name: '核心机总成', op_type: 'A', predecessors: 'P001', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P003', task_name: '液压拉伸预紧螺栓', op_type: 'A', predecessors: 'P002', std_duration: 60, time_variance: 5, work_load_sco
    re: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P004', task_name: '高频动平衡测试', op_type: 'A', predecessors: 'P003', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P011', task_name: '密封环冷冻安装', op_type: 'A', predecessors: 'P004', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P012', task_name: '陶瓷轴承预紧力设定', op_type: 'A', predecessors: 'P011', std_duration: 60, time_variance: 5, work_load_s
    core: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P013', task_name: '氦质谱检测', op_type: 'A', predecessors: 'P012', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P021', task_name: '燃烧室壳体定位', op_type: 'A', predecessors: 'P013', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P022', task_name: '燃烧室衬套安装', op_type: 'A', predecessors: 'P021', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P031', task_name: '燃烧室壳体定位', op_type: 'A', predecessors: 'P022', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P032', task_name: '燃烧室喷嘴安装', op_type: 'A', predecessors: 'P031', std_duration: 60, time_variance: 5, work_load_score
    : 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P041', task_name: '模块对接', op_type: 'A', predecessors: 'P032', std_duration: 60, time_variance: 5, work_load_score: 5, r
    ework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P042', task_name: '热间隙补偿', op_type: 'A', predecessors: 'P041', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P051', task_name: '涡轮盘装配', op_type: 'A', predecessors: 'P042', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P052', task_name: '内窥镜检测', op_type: 'A', predecessors: 'P051', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P061', task_name: '冷态旋转测试', op_type: 'A', predecessors: 'P052', std_duration: 60, time_variance: 5, work_load_score:
    5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'P062', task_name: '氦质谱检测', op_type: 'A', predecessors: 'P061', std_duration: 60, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST03' },
    { step_id: 'D001', task_name: '涡轮盘安装', op_type: 'A', predecessors: 'S004', std_duration: 30, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
    { step_id: 'D002', task_name: '核心机总成', op_type: 'A', predecessors: 'D001', std_duration: 30, time_variance: 5, work_load_score: 5,
     rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
    { step_id: 'D003', task_name: '液压拉伸预紧螺栓', op_type: 'A', predecessors: 'D002', std_duration: 30, time_variance: 5, work_load_sco
```

re: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D004', task_name: '高频动平衡测试', op_type: 'A', predecessors: 'D003', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D011', task_name: '模块对接', op_type: 'A', predecessors: 'D004', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D012', task_name: '涡轮叶片安装', op_type: 'A', predecessors: 'D011', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D013', task_name: '热间隙补偿', op_type: 'A', predecessors: 'D012', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D021', task_name: '轴承安装', op_type: 'A', predecessors: 'D013', std_duration: 40, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D022', task_name: '密封集成', op_type: 'A', predecessors: 'D021', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D031', task_name: '动力涡轮总成', op_type: 'A', predecessors: 'D022', std_duration: 40, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D041', task_name: '冷态旋转测试', op_type: 'A', predecessors: 'D031', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'D042', task_name: '氦质谱检测', op_type: 'A', predecessors: 'D041', std_duration: 30, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST05' },
{ step_id: 'G001', task_name: '壳体定位', op_type: 'A', predecessors: 'S004', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G002', task_name: '壳体安装', op_type: 'A', predecessors: 'G001', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G011', task_name: '齿轮装配', op_type: 'A', predecessors: 'G002', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G012', task_name: '核心机总成', op_type: 'A', predecessors: 'G011', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G013', task_name: '液压拉伸预紧螺栓', op_type: 'A', predecessors: 'G012', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G014', task_name: '高频动平衡测试', op_type: 'A', predecessors: 'G013', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G021', task_name: '螺旋桨轴安装', op_type: 'A', predecessors: 'G014', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G022', task_name: '陶瓷轴承预紧力设定', op_type: 'A', predecessors: 'G021', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G023', task_name: '氦质谱检测', op_type: 'A', predecessors: 'G022', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G031', task_name: '轴承安装', op_type: 'A', predecessors: 'G023', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G032', task_name: '密封集成', op_type: 'A', predecessors: 'G031', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G041', task_name: '减速箱功能总成', op_type: 'A', predecessors: 'G032', std_duration: 32, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G051', task_name: '冷态旋转测试', op_type: 'A', predecessors: 'G041', std_duration: 38, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'G052', task_name: '密封测试', op_type: 'A', predecessors: 'G051', std_duration: 38, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST06' },
{ step_id: 'F001', task_name: '齿轮箱体校准', op_type: 'A', predecessors: 'S004', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F002', task_name: '箱体组装', op_type: 'A', predecessors: 'F001', std_duration: 28, time_variance: 5, work_load_score: 5, r

ework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F003', task_name: '液压拉伸预紧螺栓', op_type: 'A', predecessors: 'F002', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F004', task_name: '高频动平衡测试', op_type: 'A', predecessors: 'F003', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F011', task_name: '主动轴齿轮安装', op_type: 'A', predecessors: 'F004', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F012', task_name: '陶瓷轴承预紧力设定', op_type: 'A', predecessors: 'F011', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F013', task_name: '氦质谱检测', op_type: 'A', predecessors: 'F012', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F021', task_name: '从动齿轮安装', op_type: 'A', predecessors: 'F013', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F031', task_name: '轴承安装', op_type: 'A', predecessors: 'F021', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F032', task_name: '密封集成', op_type: 'A', predecessors: 'F031', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F041', task_name: '附件装配', op_type: 'A', predecessors: 'F032', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F051', task_name: '齿轮啮合', op_type: 'A', predecessors: 'F041', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F052', task_name: '内窥镜检测', op_type: 'A', predecessors: 'F051', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F061', task_name: '冷态旋转测试', op_type: 'A', predecessors: 'F052', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'F062', task_name: '密封测试', op_type: 'A', predecessors: 'F061', std_duration: 28, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST04' },
{ step_id: 'B001', task_name: '燃油系统预装', op_type: 'A', predecessors: '', std_duration: 240, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST07' },
{ step_id: 'B002', task_name: '燃油系统总装', op_type: 'A', predecessors: 'B001', std_duration: 240, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST07' },
{ step_id: 'B011', task_name: '滑油系统预装', op_type: 'A', predecessors: '', std_duration: 480, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST09' },
{ step_id: 'B021', task_name: '气动系统预装', op_type: 'A', predecessors: '', std_duration: 480, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST08' },
{ step_id: 'B031', task_name: '电气系统预装', op_type: 'A', predecessors: '', std_duration: 480, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST10' },
{ step_id: 'B041', task_name: '控制系统预装', op_type: 'A', predecessors: '', std_duration: 480, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST10' },
{ step_id: 'A001', task_name: '核心机与动力涡轮对接', op_type: 'A', predecessors: 'P062;D042', std_duration: 120, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
{ step_id: 'A002', task_name: '减速齿轮箱安装', op_type: 'A', predecessors: 'A001;G052', std_duration: 120, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
{ step_id: 'A003', task_name: '附件齿轮箱安装', op_type: 'A', predecessors: 'A002;F062', std_duration: 120, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
{ step_id: 'A004', task_name: '各子系统管路连接', op_type: 'A', predecessors: 'A003', std_duration: 120, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
{ step_id: 'A005', task_name: '外部附件安装', op_type: 'A', predecessors: 'A004;B002;B011;B021;B031;B041', std_duration: 120, time_variance: 5, work_load_score: 5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
{ step_id: 'A006', task_name: '整机气密测试', op_type: 'A', predecessors: 'A005', std_duration: 120, time_variance: 5, work_load_score:

```
     5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
    { step_id: 'A007', task_name: '热试车准备', op_type: 'A', predecessors: 'A006', std_duration: 120, time_variance: 5, work_load_score: 5
    , rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST11' },
    { step_id: 'Q001', task_name: '热试车台', op_type: 'A', predecessors: 'A007', std_duration: 300, time_variance: 5, work_load_score: 5,
    rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST13' },
    { step_id: 'Q002', task_name: '性能数据采集', op_type: 'A', predecessors: 'Q001', std_duration: 300, time_variance: 5, work_load_score:
     5, rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST13' },
    { step_id: 'Q003', task_name: '最终检验', op_type: 'A', predecessors: 'Q002', std_duration: 300, time_variance: 5, work_load_score: 5,
    rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST13' },
    { step_id: 'Q004', task_name: '包装与发运', op_type: 'A', predecessors: 'Q003', std_duration: 300, time_variance: 5, work_load_score: 5
    , rework_prob: 0, required_workers: 1, required_tools: '', station: 'ST13' }
];
/* ===== File: frontend/js/editor.js ===== */
/**
 * 节点编辑器模块
 */
// 打开节点编辑模态框
function openNodeModal(node) {
    state.editingNode = node;
    document.getElementById('nodeStepId').value = node.stepId;
    document.getElementById('nodeTaskName').value = node.taskName;
    document.getElementById('nodeOpType').value = node.opType;
    document.getElementById('nodeStdDuration').value = node.stdDuration;
    document.getElementById('nodeTimeVariance').value = node.timeVariance;
    document.getElementById('nodeWorkLoad').value = node.workLoadScore;
    document.getElementById('nodeReworkProb').value = node.reworkProb;
    document.getElementById('nodeWorkers').value = node.requiredWorkers;
    document.getElementById('nodeTools').value = node.requiredTools;
    document.getElementById('nodePredecessors').value = node.predecessors;
    // 填充工位选项
    const stationSelect = document.getElementById('nodeStation');
    stationSelect.innerHTML = '<option value="">-- 请选择工位 --</option>';
    state.stations.forEach(st => {
        const opt = document.createElement('option');
        opt.value = st.id;
        opt.textContent = `${st.id} - ${st.name}`;
        opt.style.color = st.color;
        stationSelect.appendChild(opt);
    });
    stationSelect.value = node.station || '';
    document.getElementById('nodeModal').classList.add('active');
}
// 关闭节点编辑模态框
function closeNodeModal() {
    document.getElementById('nodeModal').classList.remove('active');
    state.editingNode = null;
}
// 保存节点
function saveNode() {
    if (!state.editingNode) return;
```

```
    const node = state.editingNode;
    const oldStepId = node.stepId;
    const newStepId = document.getElementById('nodeStepId').value;
    // 验证工位（必填）
    const station = document.getElementById('nodeStation').value;
    if (!station) {
        showToast('请选择工位', 'warning');
        return;
    }
    node.stepId = newStepId;
    node.taskName = document.getElementById('nodeTaskName').value;
    node.opType = document.getElementById('nodeOpType').value;
    node.stdDuration = parseFloat(document.getElementById('nodeStdDuration').value) || 30;
    node.timeVariance = parseFloat(document.getElementById('nodeTimeVariance').value) || 0;
    node.workLoadScore = parseInt(document.getElementById('nodeWorkLoad').value) || 5;
    node.reworkProb = parseFloat(document.getElementById('nodeReworkProb').value) || 0;
    node.requiredWorkers = parseInt(document.getElementById('nodeWorkers').value) || 1;
    node.requiredTools = document.getElementById('nodeTools').value;
    node.predecessors = document.getElementById('nodePredecessors').value;
    node.station = station;
    // 更新引用
    if (oldStepId !== newStepId) {
        state.nodes.forEach(n => {
            if (n.predecessors) {
                n.predecessors = n.predecessors.split(';')
                    .map(p => p.trim() === oldStepId ? newStepId : p)
                    .join(';');
            }
        });
    }
    rebuildEdges();
    closeNodeModal();
    renderCanvas();
    showToast('节点已保存', 'success');
}
// 删除节点
function deleteNode() {
    if (!state.editingNode) return;
    const nodeId = state.editingNode.id;
    const stepId = state.editingNode.stepId;
    state.nodes = state.nodes.filter(n => n.id !== nodeId);
    state.edges = state.edges.filter(e => e.from !== nodeId && e.to !== nodeId);
    // 清理引用
    state.nodes.forEach(n => {
        if (n.predecessors) {
            n.predecessors = n.predecessors.split(';')
                .filter(p => p.trim() !== stepId)
                .join(';');
        }
    });
```

```
    state.selectedNode = null;
    closeNodeModal();
    renderCanvas();
    showToast('节点已删除', 'success');
}
// 删除选中节点
function deleteSelectedNode() {
    if (!state.selectedNode) return;
    state.editingNode = state.selectedNode;
    deleteNode();
}
// 重建边
function rebuildEdges() {
    state.edges = [];
    const nodeMap = {};
    state.nodes.forEach(n => nodeMap[n.stepId] = n);
    state.nodes.forEach(node => {
        if (node.predecessors) {
            node.predecessors.split(';').forEach(predId => {
                predId = predId.trim();
                if (predId && nodeMap[predId]) {
                    state.edges.push({
                        from: nodeMap[predId].id,
                        to: node.id
                    });
                }
            });
        }
    });
}
/* ===== File: frontend/js/layout.js ===== */
/**
 * 布局模块 - 自动布局和工位相关功能
 */
// 自动布局（按依赖层级）
function autoLayout() {
    const levels = {};
    const nodeMap = {};
    state.nodes.forEach(n => {
        nodeMap[n.stepId] = n;
        levels[n.stepId] = 0;
    });
    let changed = true;
    while (changed) {
        changed = false;
        state.nodes.forEach(node => {
            if (node.predecessors) {
                node.predecessors.split(';').forEach(pred => {
                    pred = pred.trim();
                    if (pred && levels[pred] !== undefined) {
```

```
                        const newLevel = levels[pred] + 1;
                        if (newLevel > levels[node.stepId]) {
                            levels[node.stepId] = newLevel;
                            changed = true;
                        }
                    }
                });
            }
        });
    }
    const levelGroups = {};
    Object.entries(levels).forEach(([stepId, level]) => {
        if (!levelGroups[level]) levelGroups[level] = [];
        levelGroups[level].push(stepId);
    });
    const startX = 50;
    const startY = 50;
    const xGap = 160;
    const yGap = 70;
    Object.entries(levelGroups).forEach(([level, stepIds]) => {
        stepIds.forEach((stepId, idx) => {
            const node = nodeMap[stepId];
            if (node) {
                node.x = startX + parseInt(level) * xGap;
                node.y = startY + idx * yGap;
            }
        });
    });
    renderCanvas();
    showToast('自动布局完成', 'success');
}
// 按工位自动布局（UE蓝图风格）
function autoLayoutByStation() {
    // 按工位分组节点
    const stationGroups = {};
    state.nodes.forEach(node => {
        const stId = node.station || 'UNASSIGNED';
        if (!stationGroups[stId]) stationGroups[stId] = [];
        stationGroups[stId].push(node);
    });
    // 获取工位顺序
    const stationOrder = state.stations.map(s => s.id);
    stationOrder.push('UNASSIGNED');
    // 布局参数
    const containerPadding = 30;
    const containerTitleHeight = 40;
    const nodeWidth = 120;
    const nodeHeight = 50;
    const nodeGapX = 20;
    const nodeGapY = 20;
```

```
const containerGapX = 50;
const containerGapY = 30;
const maxNodesPerRow = 4;
let currentX = 50;
let currentY = 50;
let maxRowHeight = 0;
let containersPerRow = 0;
const maxContainersPerRow = 3;
stationOrder.forEach(stationId => {
    const nodes = stationGroups[stationId] || [];
    if (nodes.length === 0) return;
    const station = state.stations.find(s => s.id === stationId) || {
        id: 'UNASSIGNED',
        name: '未分配工位',
        color: '#6b7280'
    };
    // 计算容器尺寸
    const cols = Math.min(nodes.length, maxNodesPerRow);
    const rows = Math.ceil(nodes.length / maxNodesPerRow);
    const containerWidth = cols * (nodeWidth + nodeGapX) + containerPadding * 2 - nodeGapX;
    const containerHeight = rows * (nodeHeight + nodeGapY) + containerPadding * 2 + containerTitleHeight - nodeGapY;
    // 换行检查
    if (containersPerRow >= maxContainersPerRow) {
        currentX = 50;
        currentY += maxRowHeight + containerGapY;
        maxRowHeight = 0;
        containersPerRow = 0;
    }
    // 保存容器信息
    stationContainers[stationId] = {
        x: currentX,
        y: currentY,
        width: containerWidth,
        height: containerHeight,
        collapsed: false,
        color: station.color,
        name: station.name
    };
    // 布局节点
    nodes.forEach((node, idx) => {
        const col = idx % maxNodesPerRow;
        const row = Math.floor(idx / maxNodesPerRow);
        node.x = currentX + containerPadding + col * (nodeWidth + nodeGapX);
        node.y = currentY + containerTitleHeight + containerPadding + row * (nodeHeight + nodeGapY);
    });
    currentX += containerWidth + containerGapX;
    maxRowHeight = Math.max(maxRowHeight, containerHeight);
    containersPerRow++;
});
renderCanvas();
```

```
    showToast('按工位布局完成', 'success');
}
// 初始化工位筛选器
function initStationFilter() {
    const select = document.getElementById('stationFilter');
    select.innerHTML = '<option value="">全部工位</option>';
    state.stations.forEach(st => {
        const opt = document.createElement('option');
        opt.value = st.id;
        opt.textContent = st.name;
        select.appendChild(opt);
    });
}
// 按工位筛选
function filterByStation() {
    const select = document.getElementById('stationFilter');
    state.stationFilter = select.value || null;
    renderCanvas();
}
// 切换工位容器显示
function toggleContainers() {
    state.showStationContainers = document.getElementById('showContainers').checked;
    renderCanvas();
}
// 清空画布
function clearCanvas() {
    if (!confirm('确定要清空所有节点吗？')) return;
    state.nodes = [];
    state.edges = [];
    state.selectedNode = null;
    Object.keys(stationContainers).forEach(k => delete stationContainers[k]);
    renderCanvas();
    showToast('画布已清空', 'success');
}
// 加载复杂示例（基础案例，源自 data/3.csv；参数对齐实验配置 9.1.1）
function loadComplexExample() {
    loadProcessData({ name: '航空发动机整机装配流程', nodes: COMPLEX_PROCESS_NODES });
    // 全局参数（依据 9.1.1 实验配置说明）
    document.getElementById('workHoursPerDay').value = 8;     // 每日工作小时数
    document.getElementById('workDaysPerMonth').value = 22;   // 每月工作天数
    document.getElementById('numWorkers').value = 15;         // 工人数量
    document.getElementById('targetOutput').value = 10;       // 目标月产量
    // 休息规则参数
    document.getElementById('restTimeThreshold').value = 40;  // 连续工作触发阈值(分钟)
    document.getElementById('restDurationTime').value = 10;   // 时间触发休息时长(分钟)
    document.getElementById('restLoadThreshold').value = 5;   // 负荷触发阈值(REBA)
    document.getElementById('restDurationLoad').value = 10;   // 负荷触发休息时长(分钟)
    // 关键设备：试车台 x1
    state.config.criticalEquipment = {
        '试车台': 1
```

```
    };
    renderEquipmentList();
    showToast(`基础案例已加载（${COMPLEX_PROCESS_NODES.length}个节点）`, 'success');
}
// 加载流程数据
function loadProcessData(process) {
    state.nodes = process.nodes.map((n, idx) => ({
        id: 'node_' + idx,
        stepId: n.step_id,
        taskName: n.task_name,
        opType: n.op_type,
        predecessors: n.predecessors || '',
        stdDuration: n.std_duration,
        timeVariance: n.time_variance || 0,
        workLoadScore: n.work_load_score || 5,
        reworkProb: n.rework_prob || 0,
        requiredWorkers: n.required_workers || 1,
        requiredTools: Array.isArray(n.required_tools) ? n.required_tools.join(';') : (n.required_tools || ''),
        station: n.station || 'ST01',
        x: n.x || 50 + idx * 150,
        y: n.y || 100
    }));
    rebuildEdges();
    autoLayoutByStation();
}
/* ===== File: frontend/js/main.js ===== */
/**
 * 主入口模块 - 初始化
 */
document.addEventListener('DOMContentLoaded', () => {
    // 初始化各模块
    initCanvas();
    initTabs();
    initToolbox();
    initConfig();
    initStationFilter();
    initAnimations();
    // 窗口大小变化时重新计算甘特图
    window.addEventListener('resize', () => {
        if (state.simulationResult && document.getElementById('tab-gantt').classList.contains('active')) {
            updateGantt();
        }
    });
    console.log('航空发动机装配排产仿真系统 - 已加载');
});
/* ===== File: frontend/js/simulation.js ===== */
/**
 * 仿真模块
 */
// 运行仿真
```

```
function runSimulation() {
    if (state.nodes.length === 0) {
        showToast('请先添加工艺节点', 'warning');
        return;
    }
    const config = getConfigFromUI();
    const process = buildProcessDefinition();
    showLoading(true);
    fetch('/api/simulation/run', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ config, process })
    })
    .then(res => res.json())
    .then(data => {
        showLoading(false);
        if (data.success) {
            state.simulationResult = data.data;
            updateResults(data.data);
            switchTab('results');
            showToast(data.message, 'success');
        } else {
            showToast(data.message || '仿真失败', 'error');
        }
    })
    .catch(err => {
        showLoading(false);
        console.error(err);
        showToast('仿真请求失败', 'error');
    });
}
// 更新结果显示
function updateResults(result) {
    // KPI指标
    document.getElementById('kpiEngines').textContent = result.engines_completed;
    document.getElementById('kpiAchievement').textContent = (result.target_achievement_rate * 100).toFixed(1);
    document.getElementById('kpiCycleTime').textContent = (result.avg_cycle_time / 60).toFixed(1);
    document.getElementById('kpiPassRate').textContent = (result.quality_stats.first_pass_rate * 100).toFixed(1);
    const hfStats = result.human_factors_stats || {};
    document.getElementById('kpiRestTime').textContent = (hfStats.total_rest_time || 0).toFixed(0);
    document.getElementById('kpiHighIntensity').textContent = hfStats.total_high_intensity_exposure || 0;
    // 统计数据
    document.getElementById('statInspections').textContent = result.quality_stats.total_inspections;
    document.getElementById('statReworks').textContent = result.quality_stats.total_reworks;
    document.getElementById('statReworkTime').textContent = (result.quality_stats.rework_time_total || 0).toFixed(1) + '分钟';
    document.getElementById('statDuration').textContent = (result.sim_duration / 60).toFixed(1) + '小时';
    // 对比表格
    updateComparisonTable(result);
    // 瓶颈分析
    analyzeBottlenecks(result);
```

```
    // 设备详情
    updateEquipmentDetails(result);
    // 工位分析
    analyzeStationStatistics(result);
    // 工人图表
    updateWorkerChart(result);
    // 设备图表
    updateEquipmentChart(result);
    // 疲劳图表
    initFatigueSelect(result);
    updateFatigueChart();
    // 甘特图
    initGanttControls(result);
}
// 更新工人图表
function updateWorkerChart(result) {
    const workerLabels = result.worker_stats.map(w => w.resource_id);
    const workerWork = result.worker_stats.map(w => w.work_time || 0);
    const workerRest = result.worker_stats.map(w => w.rest_time || 0);
    const workerIdle = result.worker_stats.map(w => Math.max(0, result.sim_duration - (w.work_time || 0) - (w.rest_time || 0)));
    if (workerChart) workerChart.destroy();
    workerChart = new Chart(document.getElementById('workerChart'), {
        type: 'bar',
        data: {
            labels: workerLabels,
            datasets: [
                { label: '工作', data: workerWork, backgroundColor: '#3b82f6', borderRadius: 4 },
                { label: '休息', data: workerRest, backgroundColor: '#a855f7', borderRadius: 4 },
                { label: '空闲', data: workerIdle, backgroundColor: '#374151', borderRadius: 4 }
            ]
        },
        options: {
            responsive: true,
            scales: {
                x: { stacked: true, ticks: { color: '#9ca3af' }, grid: { color: '#374151' } },
                y: { stacked: true, ticks: { color: '#9ca3af' }, grid: { color: '#374151' } }
            },
            plugins: { legend: { labels: { color: '#9ca3af' } } }
        }
    });
}
// 更新设备图表
function updateEquipmentChart(result) {
    const equipLabels = result.equipment_stats.map(e => e.resource_id);
    const equipWork = result.equipment_stats.map(e => e.work_time || 0);
    const equipIdle = result.equipment_stats.map(e => Math.max(0, result.sim_duration - (e.work_time || 0)));
    if (equipmentChart) equipmentChart.destroy();
    equipmentChart = new Chart(document.getElementById('equipmentChart'), {
        type: 'bar',
        data: {
```

```
            labels: equipLabels,
            datasets: [
                { label: '使用', data: equipWork, backgroundColor: '#10b981', borderRadius: 4 },
                { label: '空闲', data: equipIdle, backgroundColor: '#374151', borderRadius: 4 }
            ]
        },
        options: {
            responsive: true,
            scales: {
                x: { stacked: true, ticks: { color: '#9ca3af' }, grid: { color: '#374151' } },
                y: { stacked: true, ticks: { color: '#9ca3af' }, grid: { color: '#374151' } }
            },
            plugins: { legend: { labels: { color: '#9ca3af' } } }
        }
    });
}
// 初始化疲劳图表选择器
function initFatigueSelect(result) {
    const fatigueSelect = document.getElementById('fatigueWorkerSelect');
    fatigueSelect.innerHTML = '<option value="all">全部工人</option>';
    const fatigueData = result.worker_fatigue_data || result.worker_stats || [];
    fatigueData.forEach(w => {
        const workerId = w.worker_id || w.resource_id;
        fatigueSelect.innerHTML += `<option value="${workerId}">${workerId}</option>`;
    });
}
// 更新对比表格
function updateComparisonTable(result) {
    const comparison = result.no_rest_comparison || {};
    const tbody = document.getElementById('comparisonBody');
    if (!comparison.engines_completed) {
        tbody.innerHTML = '<tr><td colspan="4" style="text-align: center; color: var(--text-secondary);">对比数据不可用</td></tr>';
        return;
    }
    const rows = [
        {
            label: '完成产量',
            with: result.engines_completed,
            without: comparison.engines_completed,
            unit: '台'
        },
        {
            label: '平均周期时间',
            with: (result.avg_cycle_time / 60).toFixed(1),
            without: (comparison.avg_cycle_time / 60).toFixed(1),
            unit: '小时'
        },
        {
            label: '一次通过率',
            with: (result.quality_stats.first_pass_rate * 100).toFixed(1),
```

```
            without: (comparison.quality_stats?.first_pass_rate * 100 || 0).toFixed(1),
            unit: '%'
        },
        {
            label: '总返工次数',
            with: result.quality_stats.total_reworks,
            without: comparison.quality_stats?.total_reworks || 0,
            unit: '次'
        }
    ];
    tbody.innerHTML = rows.map(row => {
        const withVal = parseFloat(row.with);
        const withoutVal = parseFloat(row.without);
        let diff = ((withVal - withoutVal) / withoutVal * 100).toFixed(1);
        let diffClass = 'neutral';
        if (row.label === '完成产量' || row.label === '一次通过率') {
            diffClass = diff > 0 ? 'positive' : (diff < 0 ? 'negative' : 'neutral');
        } else {
            diffClass = diff < 0 ? 'positive' : (diff > 0 ? 'negative' : 'neutral');
        }
        const diffIcon = diff > 0 ? '↑' : (diff < 0 ? '↓' : '→');
        diff = Math.abs(diff);
        return `
            <tr>
                <td>${row.label}</td>
                <td>${row.with} ${row.unit}</td>
                <td>${row.without} ${row.unit}</td>
                <td class="${diffClass}">${diffIcon} ${diff}%</td>
            </tr>
        `;
    }).join('');
}
/* ===== File: frontend/js/state.js ===== */
/**
 * 状态管理模块
 */
const state = {
    nodes: [],
    edges: [],
    selectedNode: null,
    editingNode: null,
    simulationResult: null,
    stations: [...DEFAULT_STATIONS],
    stationFilter: null,
    showStationContainers: true,
    layoutMode: 'normal', // 'normal' 或 'station-only'
    config: { ...DEFAULT_CONFIG }
};
// 画布相关状态
let canvas, ctx;
```

```
let canvasOffset = { x: 0, y: 0 };
let scale = 1;
let isDragging = false;
let isConnecting = false;
let connectStart = null;
let dragNode = null;
let dragOffset = { x: 0, y: 0 };
let lastMouseEvent = null;
// 图表实例
let workerChart = null;
let equipmentChart = null;
let fatigueChart = null;
/* ===== File: frontend/js/ui.js ===== */
/**
 * UI模块 - 界面交互、Tab、工具箱、加载状态
 */
// 初始化Tab
function initTabs() {
    document.querySelectorAll('.tab').forEach(tab => {
        tab.addEventListener('click', () => switchTab(tab.dataset.tab));
    });
}
// 切换Tab
function switchTab(tabId) {
    document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
    document.querySelectorAll('.tab-content').forEach(c => c.classList.remove('active'));
    const tab = document.querySelector(`.tab[data-tab="${tabId}"]`);
    const content = document.getElementById(`tab-${tabId}`);
    if (tab) tab.classList.add('active');
    if (content) content.classList.add('active');
    // 控制侧边栏显示：只在流程编辑页面显示
    const sidebar = document.getElementById('sidebar');
    if (sidebar) {
        if (tabId === 'editor') {
            sidebar.classList.add('visible');
        } else {
            sidebar.classList.remove('visible');
        }
    }
    if (tabId === 'gantt' && state.simulationResult) {
        updateGantt();
    }
    if (tabId === 'fatigue' && state.simulationResult) {
        updateFatigueChart();
    }
}
// 初始化工具箱（拖拽）
function initToolbox() {
    document.querySelectorAll('.toolbox-item').forEach(item => {
        item.addEventListener('dragstart', e => {
```

```
            e.dataTransfer.setData('opType', item.dataset.type);
        });
    });
}
// 更新统计信息
function updateStats() {
    document.getElementById('nodeCount').textContent = state.nodes.length;
    document.getElementById('edgeCount').textContent = state.edges.length;
}
// 显示/隐藏加载状态
function showLoading(show) {
    document.getElementById('loading').style.display = show ? 'flex' : 'none';
}
// 初始化配置
function initConfig() {
    renderEquipmentList();
}
// 渲染设备列表
function renderEquipmentList() {
    const list = document.getElementById('equipmentList');
    const equipment = state.config.criticalEquipment;
    list.innerHTML = Object.entries(equipment).map(([name, count]) => `
        <div style="display: flex; align-items: center; justify-content: space-between; padding: 0.5rem; background: var(--bg-secondary); b
    order-radius: 4px; margin-bottom: 0.5rem;">
            <span>${name}</span>
            <div style="display: flex; align-items: center; gap: 0.5rem;">
                <input type="number" value="${count}" min="1" style="width: 60px; padding: 0.25rem; background: var(--bg-card); border: 1px
     solid var(--border-color); border-radius: 4px; color: var(--text-primary);" onchange="state.config.criticalEquipment['${name}'] = pars
    eInt(this.value)">
                <button class="btn btn-danger" style="padding: 0.25rem 0.5rem;" onclick="removeEquipment('${name}')">×</button>
            </div>
        </div>
    `).join('');
}
// 添加设备
function addEquipment() {
    const nameInput = document.getElementById('newEquipmentName');
    const countInput = document.getElementById('newEquipmentCount');
    const name = nameInput.value.trim();
    const count = parseInt(countInput.value) || 1;
    if (!name) {
        showToast('请输入设备名称', 'warning');
        return;
    }
    if (state.config.criticalEquipment[name]) {
        showToast('设备已存在', 'warning');
        return;
    }
    state.config.criticalEquipment[name] = count;
    nameInput.value = '';
```

```
    countInput.value = '1';
    renderEquipmentList();
    showToast('设备已添加', 'success');
}
// 删除设备
function removeEquipment(name) {
    delete state.config.criticalEquipment[name];
    renderEquipmentList();
    showToast('设备已删除', 'success');
}
// 入场动画初始化：为卡片与通用容器添加淡入与上浮效果
function initAnimations() {
    const observer = new IntersectionObserver((entries) => {
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                entry.target.classList.add('animate-visible');
            }
        });
    }, { threshold: 0.1 });
    document.querySelectorAll('.kpi-card, .card').forEach(el => {
        el.classList.add('animate-initial');
        observer.observe(el);
    });
}
/* ===== File: frontend/js/utils.js ===== */
/**
 * 工具函数模块
 */
// 显示提示消息
function showToast(message, type = 'info') {
    const container = document.getElementById('toastContainer');
    const toast = document.createElement('div');
    toast.className = `toast ${type}`;
    toast.textContent = message;
    container.appendChild(toast);
    setTimeout(() => toast.classList.add('show'), 10);
    setTimeout(() => {
        toast.classList.remove('show');
        setTimeout(() => toast.remove(), 300);
    }, 3000);
}
// 颜色转换：HEX to RGBA
function hexToRgba(hex, alpha) {
    const result = /^#?([a-f\d]{2})([a-f\d]{2})([a-f\d]{2})$/i.exec(hex);
    if (!result) return `rgba(100, 100, 100, ${alpha})`;
    return `rgba(${parseInt(result[1], 16)}, ${parseInt(result[2], 16)}, ${parseInt(result[3], 16)}, ${alpha})`;
}
// 格式化时间（分钟转小时）
function formatDuration(minutes) {
    if (minutes < 60) return `${minutes.toFixed(0)}分钟`;
```

```
    return `${(minutes / 60).toFixed(1)}小时`;
}
// 格式化百分比
function formatPercent(value) {
    return `${(value * 100).toFixed(1)}%`;
}
# ===== File: config/default_config.yaml =====
# ============================================================
# 航空发动机装配排产仿真系统 - 默认配置文件
# AeroEngine Assembly Scheduling Simulation System
# 北京航空航天大学 Beihang University
# ============================================================
# 排班配置
scheduling:
  work_hours_per_day: 8        # 每日工作小时数 (1-24)
  work_days_per_month: 22      # 每月工作天数 (1-31)
  num_workers: 6               # 工人数量
  target_output: 3             # 目标月产量（台）
# 关键设备配置
# 关键设备数量有限，需要排队等待
# 普通工具（未在此列表中的）视为无限供应
critical_equipment:
  动平衡机: 2                  # 动平衡测试设备
  试车台: 1                    # 发动机试车台
  装配台: 3                    # 装配工作台
  检测台: 2                    # 检测工作台
# 休息规则配置
rest_rules:
  # 规则A: 时间触发休息
  # 当工人连续工作时间 >= 阈值时，强制休息
  time_threshold: 50           # 连续工作触发阈值（分钟）
  time_rest_duration: 5        # 时间触发休息时长（分钟）
  # 规则B: 负荷触发休息
  # 当任务负荷评分 > 阈值时，任务后休息
  load_threshold: 7            # 负荷触发阈值（REBA评分 1-10）
  load_rest_duration: 3        # 负荷触发休息时长（分钟）
# 仿真选项
simulation:
  pipeline_mode: true          # 流水线模式（多台并行）
  random_seed: null            # 随机种子（null表示随机）
  max_engines: 10              # 最大仿真发动机数量
# 工艺节点默认值
node_defaults:
  std_duration: 30             # 默认标准工时（分钟）
  time_variance: 5             # 默认时间方差
  work_load_score: 5           # 默认负荷评分
  rework_prob: 0.05            # 默认返工概率（仅M类型）
  required_workers: 1          # 默认所需工人数
# 操作类型定义
op_types:
```

```
  H:
    name_zh: 取/放
    name_en: Handling
    color: "#3B82F6"
    icon: "  "
    description: 零部件的取放操作
  A:
    name_zh: 装配
    name_en: Assembly
    color: "#10B981"
    icon: "  "
    description: 装配组合操作
  M:
    name_zh: 测量
    name_en: Measurement
    color: "#F59E0B"
    icon: "  "
    description: 测量检验操作，可触发返工
    can_rework: true
  T:
    name_zh: 工具操作
    name_en: Tooling
    color: "#8B5CF6"
    icon: "    "
    description: 使用工具的操作
  D:
    name_zh: 数据记录
    name_en: Data Recording
    color: "#6B7280"
    icon: "  "
    description: 记录数据的操作
# 甘特图显示配置
gantt:
  event_colors:
    NORMAL: "#3B82F6"          # 正常工作 - 蓝色
    REST: "#6B7280"            # 休息 - 灰色
    REWORK: "#EF4444"          # 返工 - 红色
    WAITING: "#F59E0B"         # 等待 - 橙色
  default_view_days: 3         # 默认显示天数
  hour_width_px: 60            # 每小时像素宽度
# 导出配置
export:
  csv_encoding: utf-8-sig      # CSV编码（支持Excel）
  json_ensure_ascii: false     # JSON保留中文
  date_format: "%Y-%m-%d %H:%M:%S"
# 系统信息
system:
  version: "1.0.0"
  author: "北京航空航天大学"
  project: "航空发动机装配排产仿真系统"
```